

第7章 图

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历

7.4 最小生成树

7.5 活动网络

7.5.1 拓扑排序

7.5.2 关键路径

7.6 最短路径

回顾

□ 两种常用的活动网络(Activity Network)

① **AOV网(Activity On Vertices)**—用**顶点**表示活动的网络

AOV网定义：若用有向图表示一个工程，在图中用**顶点**表示活动，用弧表示活动间的优先关系。 v_i 必须先于活动 v_j 进行。则这样的有向图叫做用顶点表示活动的网络。

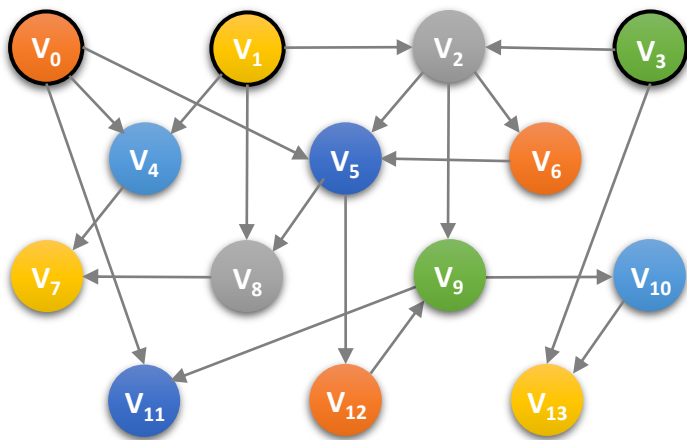
② **AOE网(Activity On Edges)**—用**边**表示活动的网络

AOE网定义：如果在无环的**带权**有向图中，用**有向边**表示一个工程中的活动，用边上权值表示活动持续时间，用顶点表示事件，则这样的有向图叫做用边表示活动的网络。

回顾

拓扑排序

1. 在有向图中选一个没有前驱的顶点并输出
2. 从图中删除该顶点和所有以它为尾的弧
3. 重复上述两步，直至全部顶点均已输出；或者当图中不存在无前驱的顶点为止



回顾

AOE网

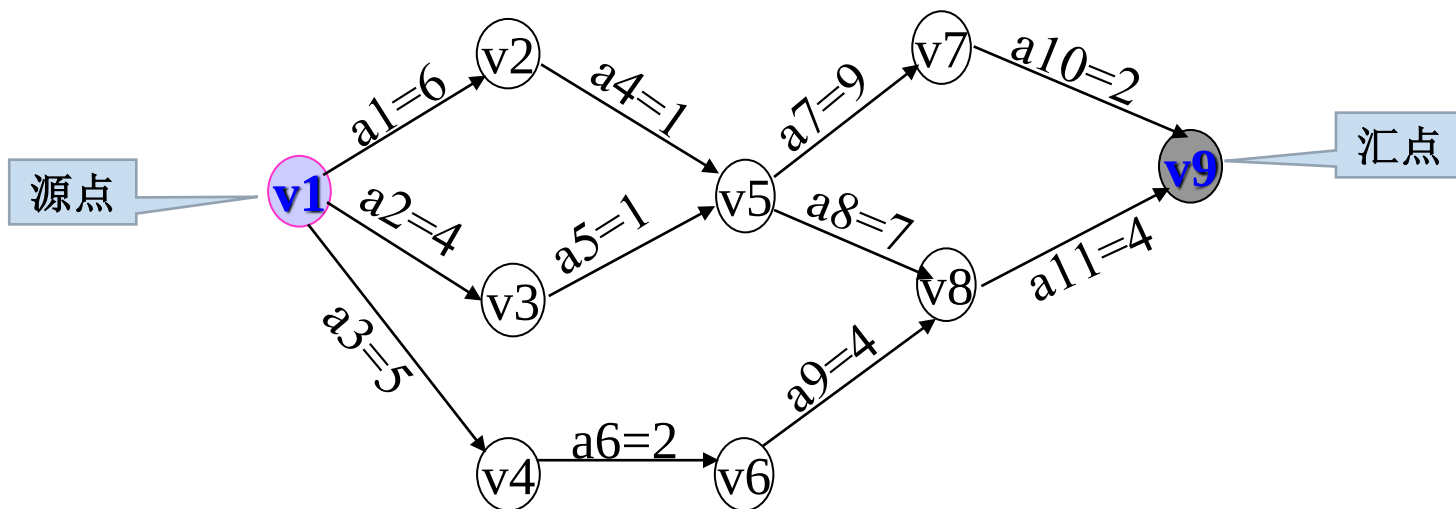
事件 V1 (源点) ——表示整个工程开始

事件V9 (汇点) ——表示整个工程结束

有向边 (弧) ---表示活动

弧的权值-----表示该活动持续时间

每个事件表示在它之前的活动已经结束，在它之后的活动可以开始。



7.5.2 关键路径

□ 关键路径

对整个工程和系统，通常关心三个方面的问题

(1) 工程能否顺利进行

对AOV网进行拓扑排序

(2) 完成整项工程至少需要多少时间？

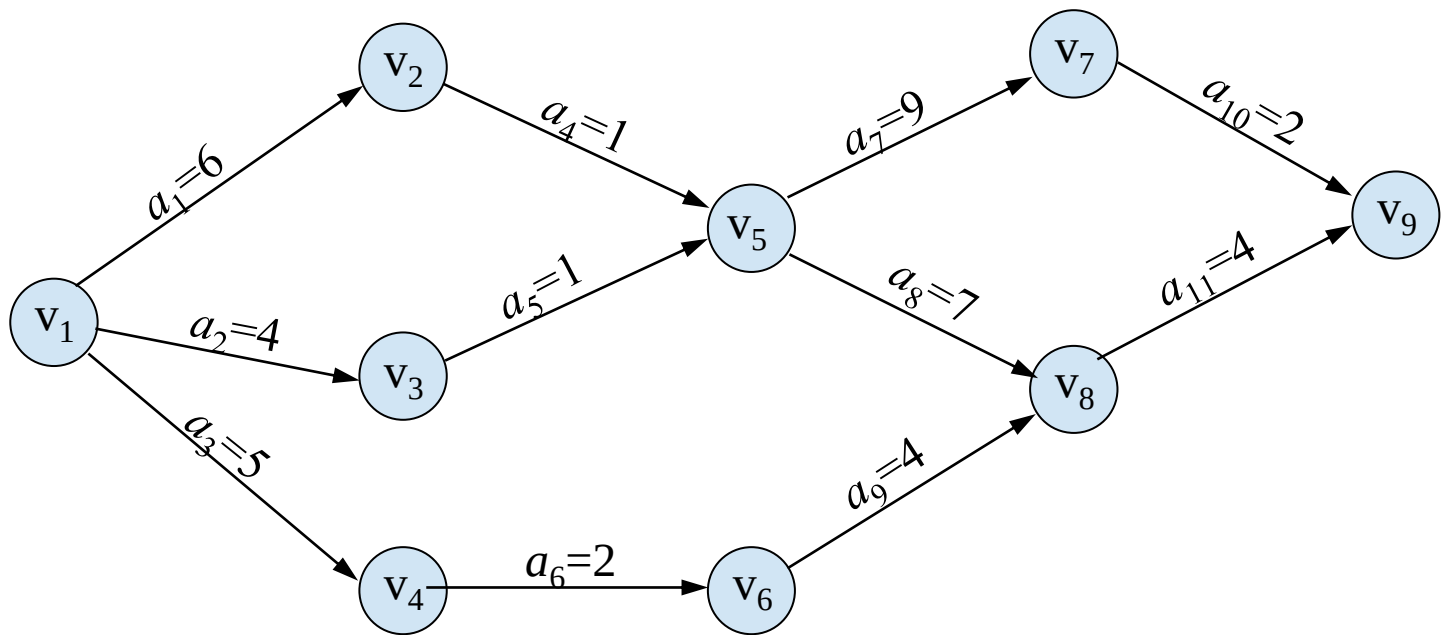
对AOE网求关键路径

(3) 哪些活动是影响工程进度的关键？

对AOE网求关键路径

7.5.2 关键路径

□ 关键路径



完成工程的最短时间是从开始点到完成点的**最长路径**的长度。
路径长度最长的路径叫做**关键路径**。

从 v_1 到 v_9 的最长路径是 $(v_1, v_2, v_5, v_8, v_9)$ ，路径长度是 18。

7.5.2 关键路径

- 事件 V_i 的最早可能开始时间 $Ve[i]$: 是从源点 V_0 到顶点 V_i 的最长路径长度。
- 事件 V_i 的最迟允许开始时间 $VI[i]$: 是在保证汇点 V_{n-1} 在 $Ve[n-1]$ 时刻完成的前提下, 事件 V_i 的允许的最迟开始时间。
- 活动 a_k 的最早可能开始时间 $e[k]$: 设活动 a_k 在边 $\langle V_i, V_j \rangle$ 上, 则 $e[k]$ 是从源点 V_0 到顶点 V_i 的最长路径长度。因此, $e[k] = Ve[i]$ 。
- 活动 a_k 的最迟允许开始时间 $l[k]$: $l[k]$ 是在不会引起时间延误的前提下, 该活动允许的最迟开始时间。 $l[k] = VI[j] - dur(\langle i, j \rangle)$ 。

其中, $dur(\langle i, j \rangle)$ 是完成 a_k 所需的时间。

- 时间余量 $l[k] - e[k]$: 表示活动 a_k 的最早可能开始时间和最迟允许开始时间的的时间余量。 $l[k] == e[k]$ 表示活动 a_k 是没有时间余量的关键活动。

为找出关键活动, 要求各活动的 $e[k]$ 与 $l[k]$, 以判别是否 $l[k] == e[k]$ 。

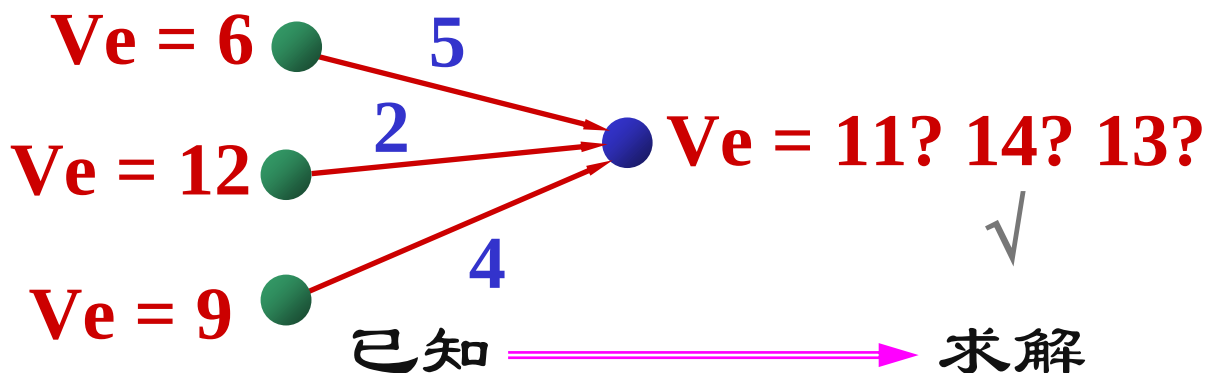
7.5.2 关键路径

- 为求得 $e[k]$ 与 $l[k]$, 需要先求得从源点 V_0 到各个顶点 V_i 的 $Ve[i]$ 和 $Vl[i]$ 。
- 求 $Ve[i]$ 的递推公式

从 $Ve[0] = 0$ 开始, 向前递推:

$$Ve[j] = \max_i \{ Ve[i] + dur(<V_i, V_j>) \},$$

其中, $<V_i, V_j> \in S2, j = 1, 2, \dots, n-1$ 。 $S2$ 是所有指向 V_j 的有向边 $<V_i, V_j>$ 的集合。



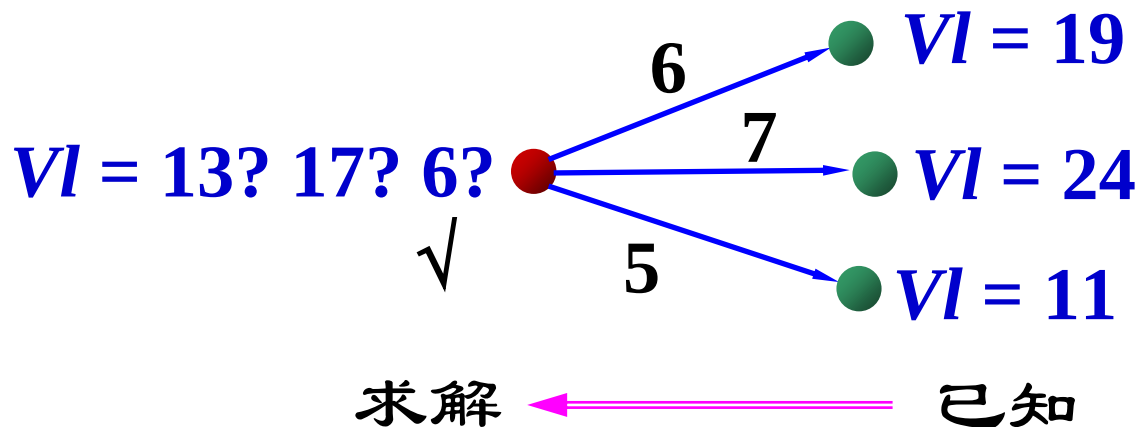
7.5.2 关键路径

- 求 $VI[i]$ 的递推公式:

从 $VI[n-1] = Ve[n-1]$ 开始, 反向递推:

$$VI[j] = \min_k \{ VI[k] - dur(< V_j, V_k >) \},$$

其中, $< V_j, V_k > \in S1, j = n-2, n-3, \dots, 0$ 。 $S1$ 是所有源自 V_j 的有向边 $< V_j, V_k >$ 的集合。



这两个递推公式的计算必须分别在**拓扑有序**及**逆拓扑有序**的前提下进行。

7.5.2 关键路径

- 设活动 a_k ($k=1, 2, \dots, e$)在带权有向边 $\langle V_i, V_j \rangle$ 上, 其持续时间用 $\text{dur}(\langle V_i, V_j \rangle)$ 表示, 则有

$$e[k] = Ve[i];$$

$$l[k] = Vl[j] - \text{dur}(\langle V_i, V_j \rangle) ; k = 1, 2, \dots, e。$$

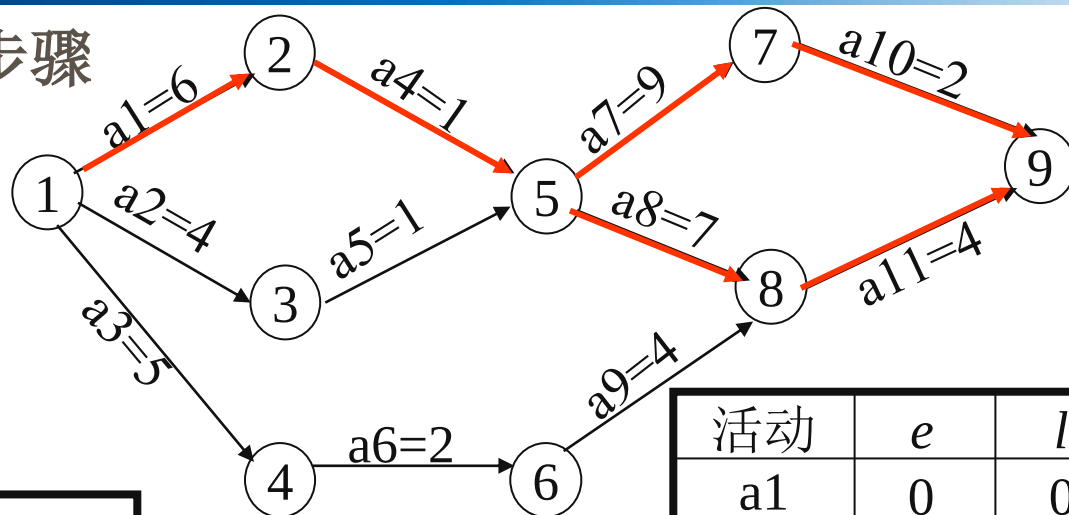
这样就得到计算关键路径的算法。

- 为了简化算法, 假定在求关键路径之前已经对各顶点实现了拓扑排序, 并按拓扑有序的顺序对各顶点重新进行了编号。

7.5.2 关键路径

● 求关键路径步骤

- ◆ 求 $ve(i)$
- ◆ 求 $vl(j)$
- ◆ 求 $e(i)$
- ◆ 求 $l(i)$
- ◆ 计算 $l(i)-e(i)$



顶点	ve	vl
V1	0	0
V2	6	6
V3	4	6
V4	5	8
V5	7	7
V6	7	10
V7	16	16
V8	14	14
V9	18	18

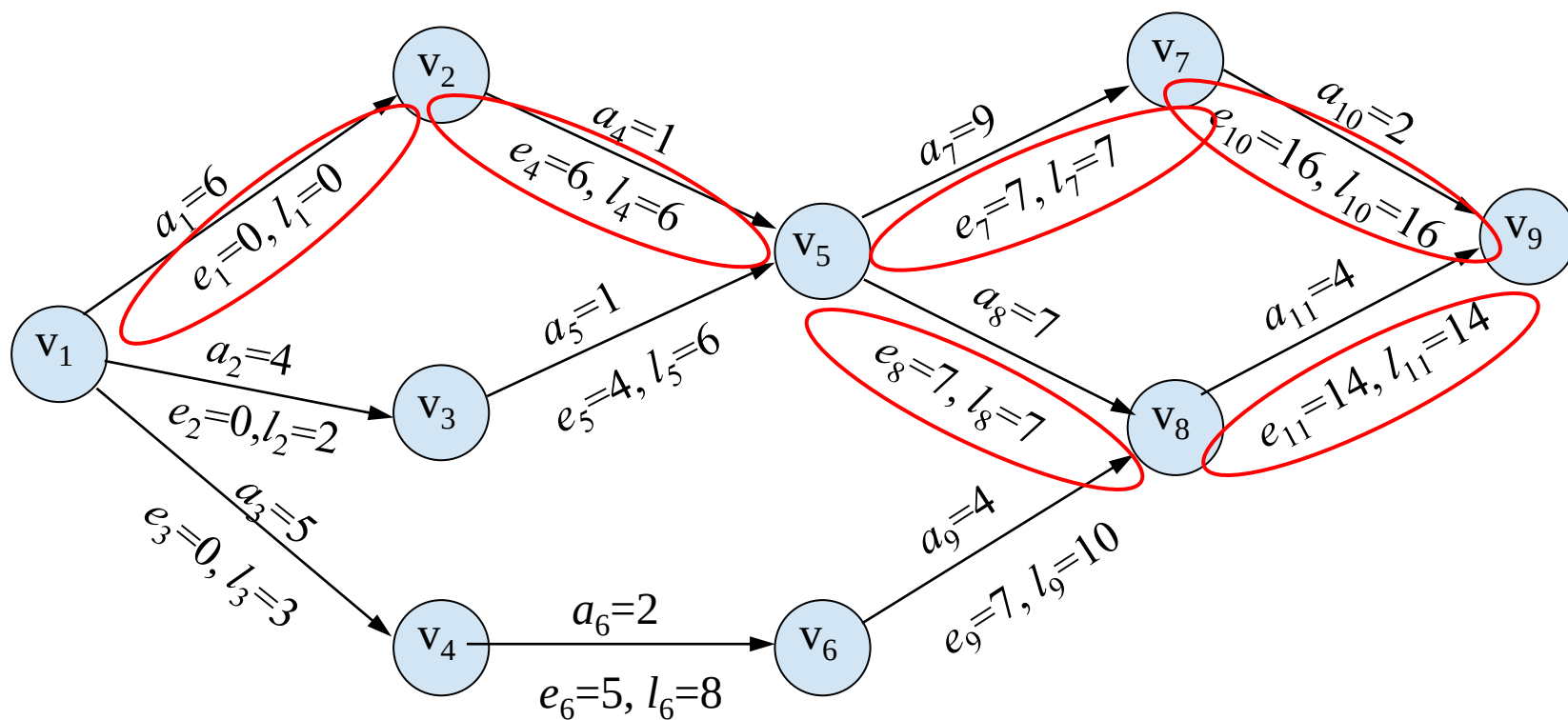
$$e[k] = Ve[i]$$

$$l[k] = Vl[j] - dur(< V_i, V_j >)$$

活动	e	l	$l-e$
a1	0	0	0 ✓
a2	0	2	2
a3	0	3	3
a4	6	6	0 ✓
a5	4	6	2
a6	5	8	3
a7	7	7	0 ✓
a8	7	7	0 ✓
a9	7	10	3
a10	16	16	0 ✓
a11	14	14	0 ✓

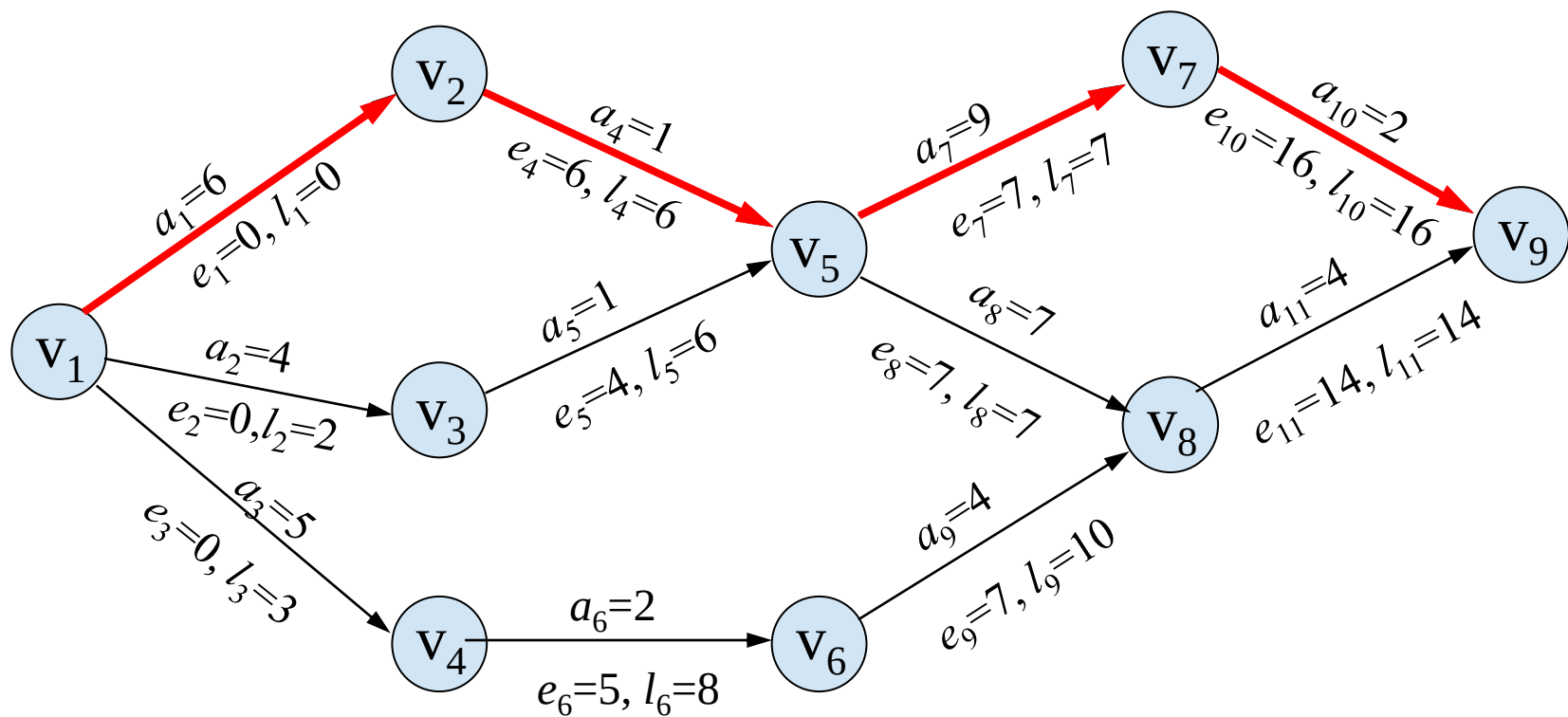
7.5.2 关键路径

□ 关键路径



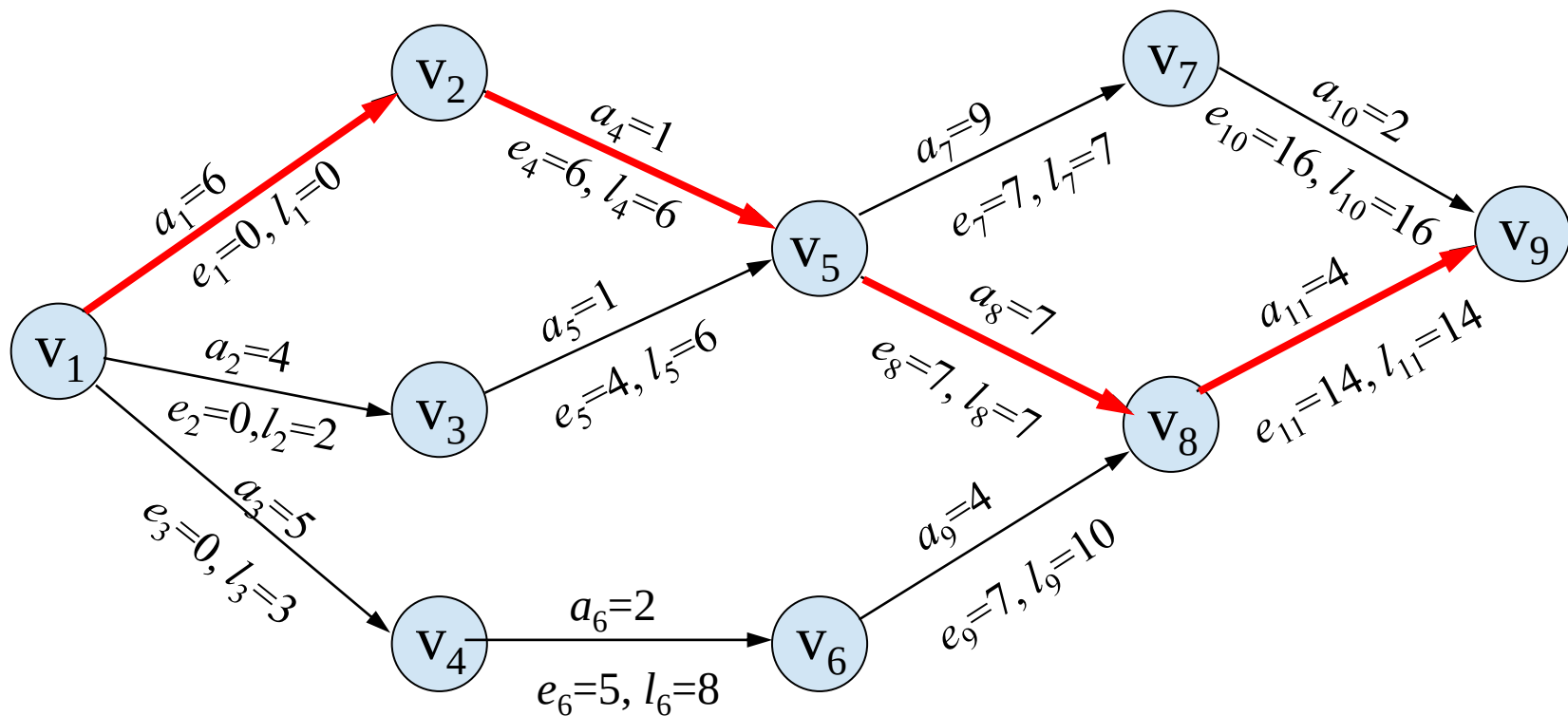
7.5.2 关键路径

□ 关键路径



7.5.2 关键路径

□ 关键路径



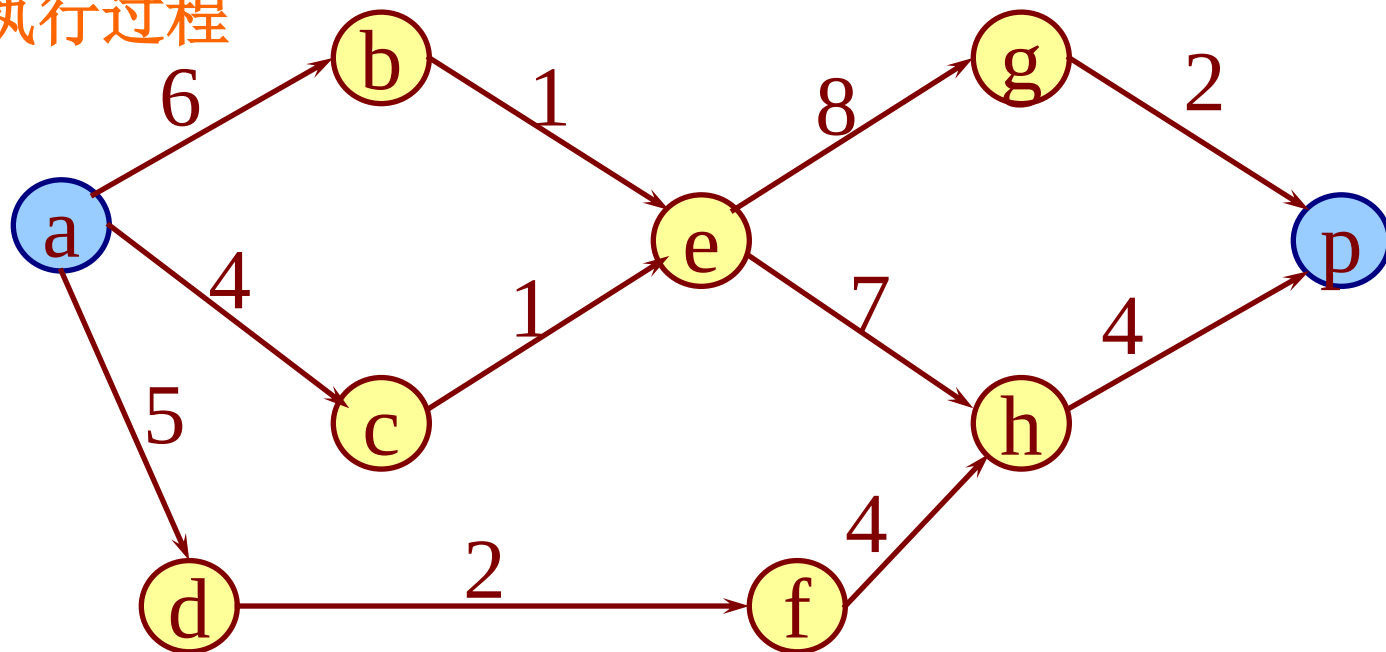
7.5.2 关键路径

□ 关键路径算法思想

- 1) 输入 e 条弧 (i, j) ，建立AOE网的存储结构。
- 2) 从源点 v_0 出发，令 $ve[0]=0$ 按拓扑有序求其余各顶点的最早发生时 $ve[i]$ ($1 \leq i \leq n-1$)。如果得到的拓扑有序序列中顶点个数小于网中顶点数 n ，则说明网中存在环，不能求关键路径，算法终止；否则执行步骤（3）。
- 3) 从汇点 v_n 出发，令 $vl[n-1]= ve[n-1]$ ，按逆拓扑有序求其余各顶点的最迟发生时间 $vl[i]$ ($n-2 \geq i \geq 2$)；
- 4) 根据各顶点的 ve 和 vl 值，求每条弧 s 的最早开始时间 $e(s)$ 和最迟开始时间 $l(s)$ 。若某条弧满足条件 $e(s)=l(s)$ ，则为关键活动。

7.5.2 关键路径

算法的执行过程



	a	b	c	d	e	f	g	h	p
Ve	0	6	4	5	7	7	15	14	18
VI	0	6	6	8	7	10	16	14	18

拓扑有序序列: **a - d - f - c - b - e - h - g - p**



7.5.2 关键路径

	a	b	c	d	e	f	g	h	p
<i>Ve</i>	0	6	4	5	7	7	15	14	18
<i>VI</i>	0	6	6	8	7	10	16	14	18

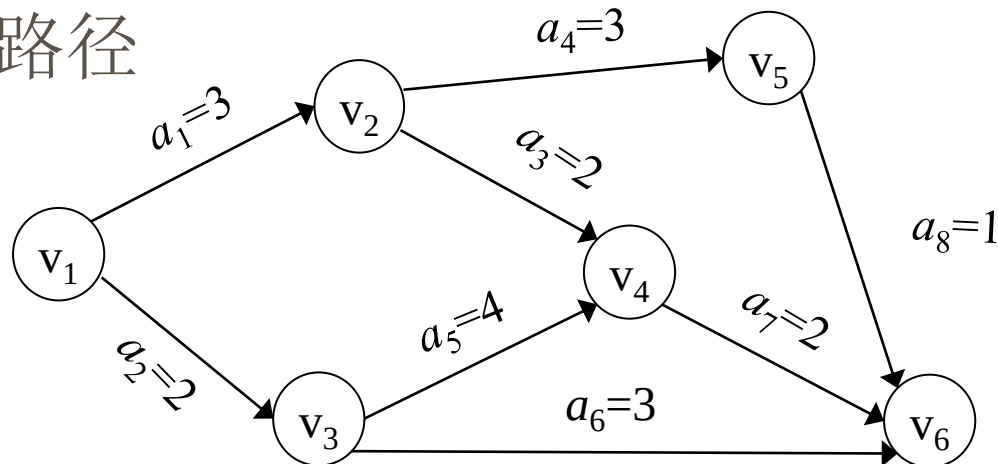
	ab	ac	ad	be	ce	df	eg	eh	fh	gp	hp
权	6	4	5	1	1	2	8	7	4	2	4
e	0	0	0	6	4	5	7	7	7	15	14
l	0	2	3	6	6	8	8	7	10	16	14
											

7.5.2 关键路径

```
Status TopologicalSort ( ALGraph G, Stack &T) {  
    FindInDegree(G, indegree); //计算各顶点的入度    T为拓扑排序顶点栈  
    InitStack(S); count=0; ve[0 .. G.vexnum-1] = 0,  
    for ( i=0; i<G.vexnum; ++i) if (!indegree[i]) Push(S, i); //入度为零的顶点入栈S  
    while (!EmptyStack(S)) {          //S为入度为零顶点栈  
        Pop(S, j); Push(T, j); ++count; printf(v);  
        for (p = G. vertices[j]. firstarc; p; p = p->nextarc){  
            k = p->adjvex;  
            if (--indegree[k] == 0) Push(S, k); // 弧头顶点的入度减1, 为零的顶点入栈S  
            if (ve[j] + *(p->info) > ve[k]) ve[k] = ve[j] + * (p->info);  
        } //for  
    } //while  
    if (count<G.vexnum) printf(“图中有回路” ); return ERROR;  
    else return OK;  
}
```

7.5.2 关键路径

□ 关键路径 例



T

V6
V5
V4
V3
V2
V1

0	V1		→	2	2		→	1	3	^
1	V2		→	4	3		→	3	2	^
2	V3		→	5	3		→	3	4	^
3	V4		→	5	2	^				
4	V5		→	5	1	^				
5	V6	^								

拓扑排序: V1 V2 V5 V3 V4 V6

$ve(\text{源点}) = 0$

$ve(j) = \text{Max}\{ve(i) + \text{dut}(<i, j>)\}$

if $(ve[j] + *(p \rightarrow \text{info}) > ve[k])$ $ve[k] = ve[j] + *(p \rightarrow \text{info});$

	V1	V2	V3	V4	V5	V6
Ve	0	3	2	6	6	8

7.5.2 关键路径

Status CriticalPath (ALGraph G) { //输出G的各项关键活动。

if (!TopologicalOrder (G, T)) return ERROR;

vl[0..G.vexnum-1]=ve[G.vexnum-1]; //初始化事件最迟发生时间

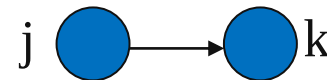
while (! StackEmpty (T)) //按拓扑逆序求各顶点的vl 值

for (Pop(T,j), p=G.vertices[j].firstarc; p; p=p->nextarc) {

k=p->adjvex; dut=*(p->info); //dut<j, k>

if (vl[k] - dut < vl[j]) vl[j] = vl[k] - dut;

} // for 以j为弧尾的弧的弧头k



for (j=0; j < G.vexnum; ++j) //求ee、el和关键活动

for(p=G.vertices[j].firstarc; p; p=p->nextarc){

k = p->adjvex ; dut = *(p->info) ;

ee = ve[j] ; el=vl[k]-dut ;

tag = (ee == el) ? '*' : ' ' ;

printf (j, k, dut, ee, el, tag) ;

// 输出关键活动



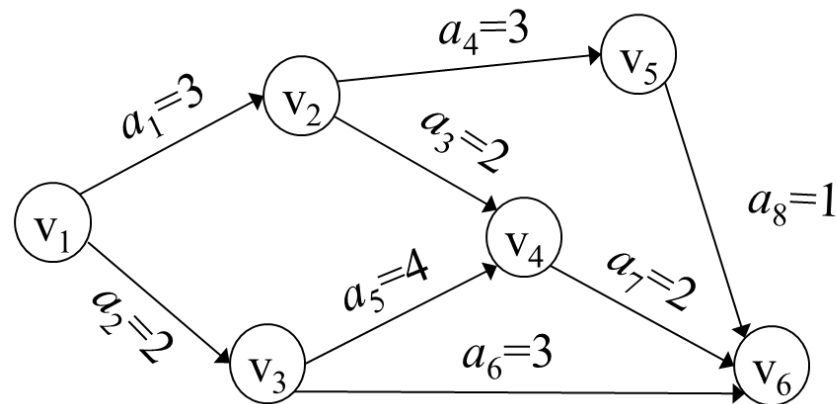
} // for

}//CriticalPath

7.5.2 关键路径

□ 关键路径

$vl(\text{汇点}) = ve(\text{汇点});$
 $vl(i) = \text{Min} \{ vl(j) - \text{dut}(\langle i, j \rangle) \}$



```

while ( ! StackEmpty (T) )
    for (Pop(T,j), p=G.vertices[j].firstarc; p; p=p->nextarc) {
        k=p->adjvex; dut=*(p->info); //dut<j, k>
        if ( vl[k] - dut < vl[j] ) vl[j] = vl[k] - dut;
    } // for
    
```

	V1	V2	V3	V4	V5	V6
Ve[]	0	3	2	6	6	8
Vl[]	0	6	4	6	7	8

T

V6
V5
V4
V3
V2
V1

7.5.2 关键路径

□ 关键路径

	V1	V2	V3	V4	V5	V6
$Ve[]$	0	3	2	6	6	8
$Vl[]$	0	4	2	6	7	8

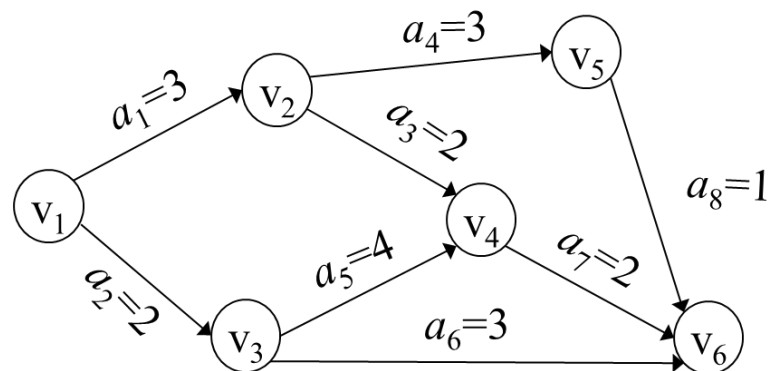
$$e(s) = Ve(i)$$

$$l(s) = Vl(j) - dut(<i, j>)$$

```

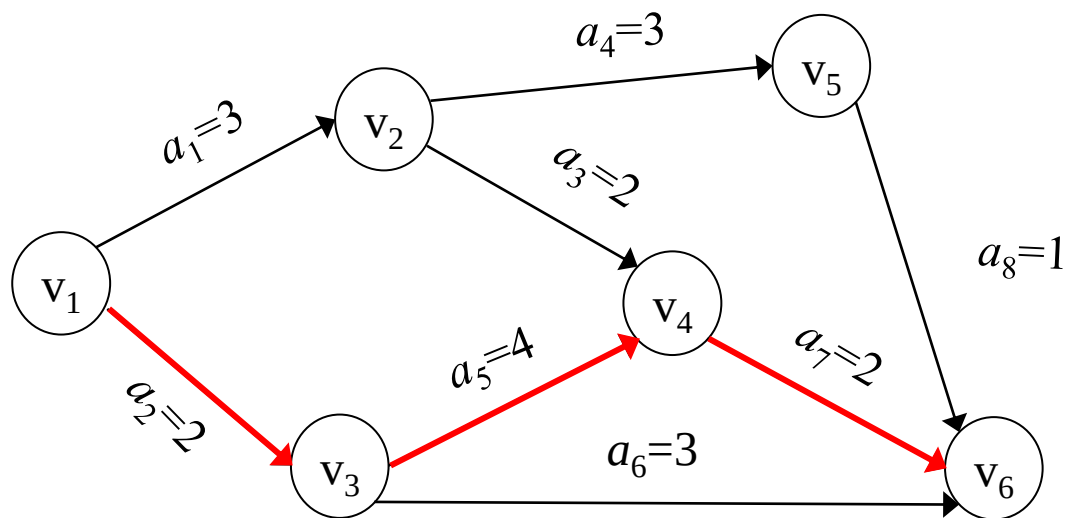
for (j=0; j < G.vexnum; ++j )
    for( p=G.vertices[j].firstarc; p; p=p->nextarc){
        k = p->adjvex ; dut = *(p->info) ;
        ee = ve[j] ;      el=vl[k]-dut ;
        tag = ( ee == el ) ? '*' : ' ' ;
        printf ( j, k, dut, ee, el, tag ) ;
    } // for
    
```

	a1	a2	a3	a4	a5	a6	a7	a8
$e[]$	0	0	3	3	2	2	6	6
$l[]$	1	0	4	4	2	5	6	7
$l-e$	1	0	1	1	0	3	0	1



7.5.2 关键路径

□ 关键路径 例



7.5.2 关键路径

□ 关键路径

- 求 $ve(i)$

$$ve(\text{源点}) = 0$$

$$ve(j) = \text{Max}\{ ve(i) + \text{dut}(<i, j>) \}$$

- 求 $vl(j)$

$$vl(\text{汇点}) = ve(\text{汇点});$$

$$vl(i) = \text{Min}\{ vl(j) - \text{dut}(<i, j>) \}$$

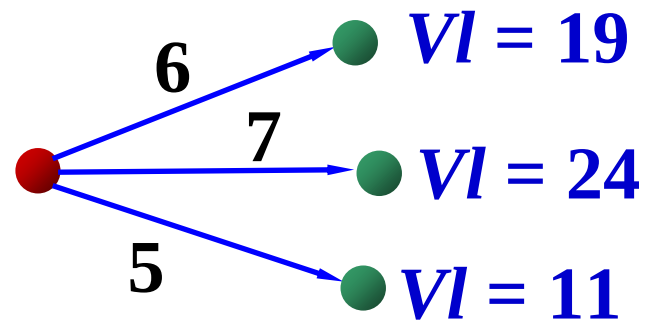
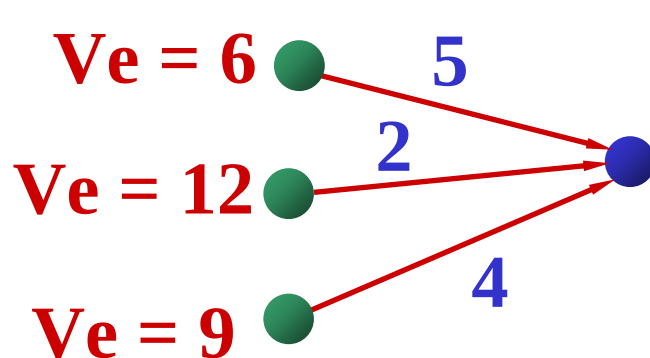
- 求 $e(i)$

$$e(s) = Ve(i)$$

- 求 $l(i)$

$$l(s) = Vl(j) - \text{dut}(<i, j>)$$

- 计算 $l(i) - e(i)$



第7章 图

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历与连通性

7.4 最小生成树

7.5 活动网络

7.6 最短路径

最短路径



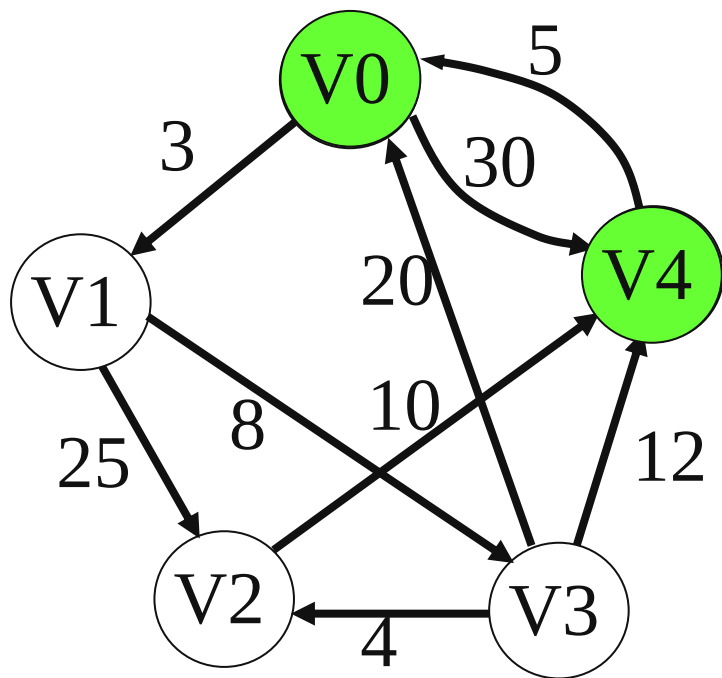
最短路径

□ 问题抽象：在带权有向图中A点（源点）到达B点（终点）的多条路径中，寻找一条各边权值之和最小的路径，即最短路径。

(注：最短路径与最小生成树不同，路径上不一定包含n个顶点)

- 旅游者:最经济的出行路线
- 路由器: 最快地将数据包传送到目标位置
- 路径规划:多边形区域内的自主机器人
-

最短路径



0	1	2	3	4	
0	3	∞	∞	30	0
∞	0	25	8	∞	1
∞	∞	0	∞	10	2
20	∞	4	0	12	3
5	∞	∞	∞	0	4

从V0到V4共有三条路径：

$\{V0, V4\}$, $\{V0, V1, V3, V4\}$, $\{V0, V1, V2, V4\}$

30

23

38

最短路径

一顶点到其余各
顶点

一、单源最短路径——用 Dijkstra (迪杰斯特拉)
算法

二、所有顶点间的最短路径——用 Floyd (弗洛伊德)
算法

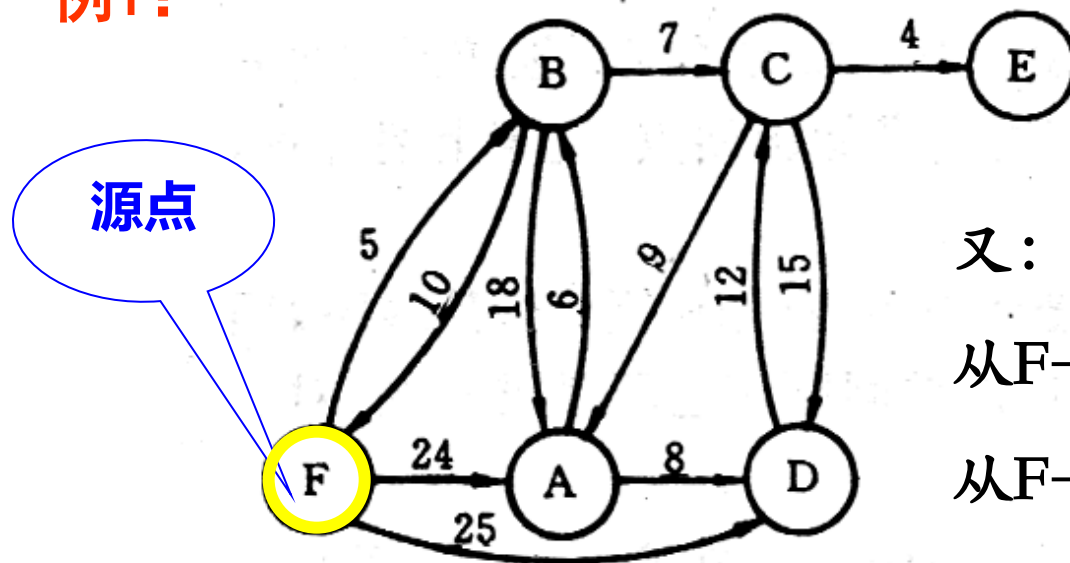
任意两顶点之间

最短路径——迪杰斯特拉算法

目的： 设一有向图 $G = (V, E)$ ，已知各边的权值，以某指定点 v_0 为源点，求从 v_0 到图的其余各点的最短路径。限定各边上的权值大于或等于0。

最短路径——迪杰斯特拉算法

例1:



又:

从 $F \rightarrow B$ 的最短路径是哪条?

从 $F \rightarrow C$ 的最短路径是哪条?

从 $F \rightarrow A$ 的路径有4条:

- ① $F \rightarrow A$: 24
- ② $F \rightarrow B \rightarrow A$: $5 + 18 = 23$
- ③ $F \rightarrow B \rightarrow C \rightarrow A$: $5 + 7 + 9 = 21$
- ④ $F \rightarrow D \rightarrow C \rightarrow A$: $25 + 12 + 9 = 36$

最短路径

●为求得这些最短路径，Dijkstra提出按路径长度的递增次序，逐步产生最短路径的算法。首先求出长度最短的一条最短路径，再参照它求出长度次短的一条最短路径，依次类推，直到从顶点 v 到其它各顶点的最短路径全部求出为止。

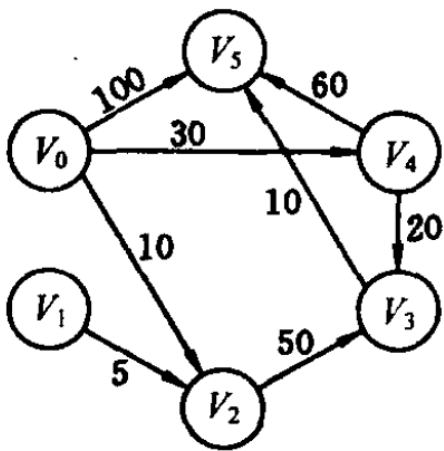


图 7.34 带权有向图 G_6

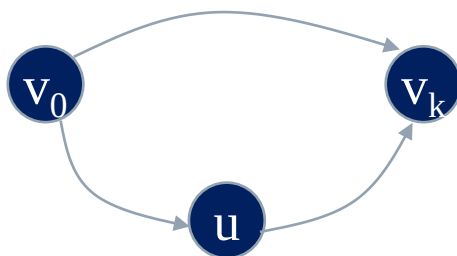
始点	终点	最短路径	路径长度
v_0	v_1	无	
	v_2	(v_0, v_2)	10
	v_3	(v_0, v_4, v_3)	50
	v_4	(v_0, v_4)	30
	v_5	(v_0, v_4, v_3, v_5)	60

图 7.35 有向图 G_6 中从 v_0 到其余各点的最短路径

最短路径

□ 最短路径

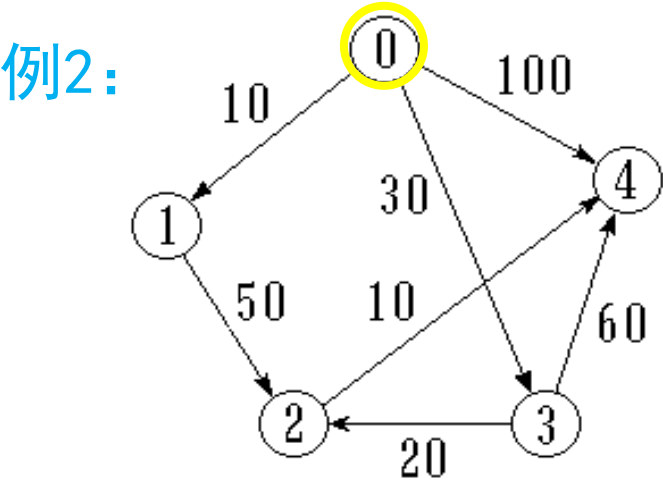
Dijkstra算法思想:



如果: $(v_0, u) + (u, v_k) < (v_0, v_k)$

则: 以路径 (v_0, u, v_k) 代替 (v_0, v_k)

最短路径——迪杰斯特拉算法



(a) 带权有向图

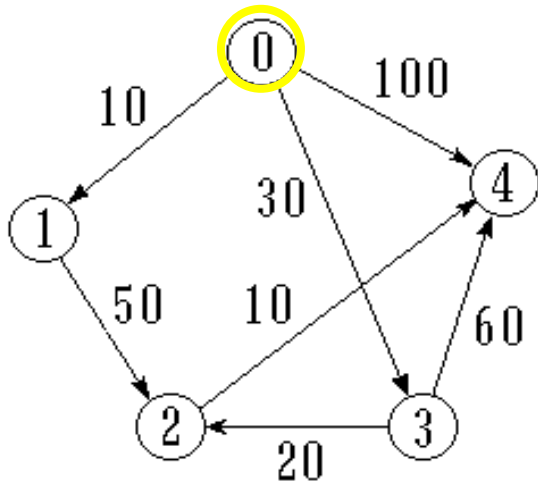
	0	1	2	3	4
0	0	10	∞	30	100
1	∞	0	50	∞	∞
2	∞	∞	0	∞	10
3	∞	∞	20	0	60
4	∞	∞	∞	∞	0

(b) 邻接矩阵

源点	终点	最 短 路 径	路径长度
V_0	V_1	(V_0, V_1)	10
	V_2	(V_0, V_1, V_2) (V_0, V_3, V_2)	60 50
	V_3	(V_0, V_3)	30
	V_4	(V_0, V_4) (V_0, V_3, V_4) (V_0, V_1, V_2, V_4) (V_0, V_3, V_2, V_4)	100 90 70 60

最短路径——迪杰斯特拉算法

例2:



(a) 带权有向图

	0	1	2	3	4
0	0	10	∞	30	100
1	∞	0	50	∞	∞
2	∞	∞	0	∞	10
3	∞	∞	20	0	60
4	∞	∞	∞	∞	0

(b) 邻接矩阵

源点	终点	最 短 路 径	路径长度
V_0	V_1	(V_0, V_1)	10
	V_2	(V_0, V_1, V_2) (V_0, V_3, V_2)	60 50
	V_3	(V_0, V_3)	30
	V_4	(V_0, V_4) (V_0, V_3, V_4) (V_0, V_1, V_2, V_4) (V_0, V_3, V_2, V_4)	100 90 70 60

最短路径——迪杰斯特拉算法

算法思想：

(1) 先找出从源点 v_0 到各终点 v_k 的直达路径 (v_0, v_k) , 即通过一条弧到达的路径;

(2) 从这些路径中找出一条长度最短的路径 (v_0, u) , 然后对其余各条路径进行适当调整:

若在图中存在弧 (u, v_k) , 且 $(v_0, u) + (u, v_k) < (v_0, v_k)$, 则以路径 (v_0, u, v_k) 代替 (v_0, v_k) 。其中, (v_0, u) 为上次求得的 v_0 到 u 最短路径;

(3) 在调整后的各条路径中, 再找长度最短的路径, 依此类推。

总之, 按路径“长度” 递增的次序来逐步产生最短路径

最短路径——迪杰斯特拉算法

(1) 设 $A[n][n]$ 为有向网的带权邻接矩阵, $A[i][j]$ 表示弧 (v_i, v_j) 的权值, S 为已找到从源点 v_0 出发的最短路径的终点集合, 它的初始状态为 $\{v_0\}$, 辅助数组 $D[n]$ 为源点 v_0 到各终点当前找到的最短路径的长度, 它的初始值为: $D[i] = A[v_0, i]$ //即邻接矩阵中第 v_0 行的权值;

最短路径——迪杰斯特拉算法

(2) 选择j, 使得

$$D[j] = \min\{D[i] \mid v_i \in V-S\}$$

/* v_i 是S集之外的顶点, j是当前离源点距离最短的顶点, $D[j]$ 是从源点 v_0 到S集合之外所有顶点的路径中距离最短的一条*/

$S = S \cup \{v_j\}$ //将j加入S集

最短路径——迪杰斯特拉算法

(3) 对于所有不在S中的终点 v_k , 若

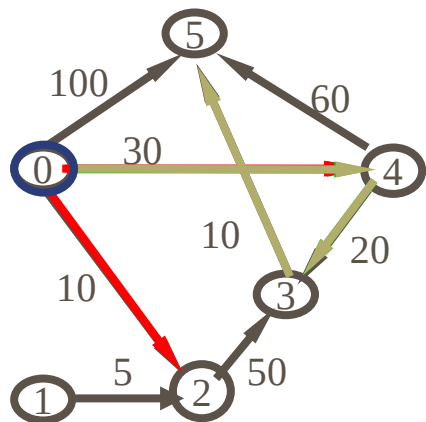
$$D[j] + A[j,k] < D[k] \quad // \text{ 即 } (v_0, j) + (j, k) < (v_0, k)$$

则修改D[k]为: $D[k] = D[j] + A[j,k]$

$D[j]$ 为上次求得从源点到 v_j 点的最短路径, v_j 为上次求得最短路径的点。

(4) 重复操作(2)、(3)共n-1次, 由此求得从 v_0 到各终点的最短路径。

最短路径——迪杰斯特拉算法



	0	1	2	3	4	5
0	0	∞	10	∞	30	100
1	∞	0	5	∞	∞	∞
2	∞	∞	0	50	∞	∞
3	∞	∞	∞	0	∞	10
4	∞	∞	∞	20	0	60
5	∞	∞	∞	∞	∞	0

S之外的当前最短路径之顶点

终点	D[w]	从 v_0 到各终点的D值和最短路径			
v_1	∞	∞	∞	∞	∞
v_2	10 { v_0, v_2 }			10 { v_0, v_2 }	
v_3	∞	60 { v_0, v_2, v_3 }	50 { v_0, v_4, v_3 }	50 { v_0, v_4, v_3 }	
v_4	30 { v_0, v_4 }	30 { v_0, v_4 }		30 { v_0, v_4 }	
v_5	100 { v_0, v_5 }	100 { v_0, v_5 }	90 { v_0, v_4, v_5 }	60 { v_0, v_4, v_3, v_5 }	
v_j	v_2	v_4	v_3	v_5	
S	{ v_0, v_2 }	{ v_0, v_2, v_4 }	{ v_0, v_2, v_4, v_3 }	{ v_0, v_2, v_4, v_3, v_5 }	

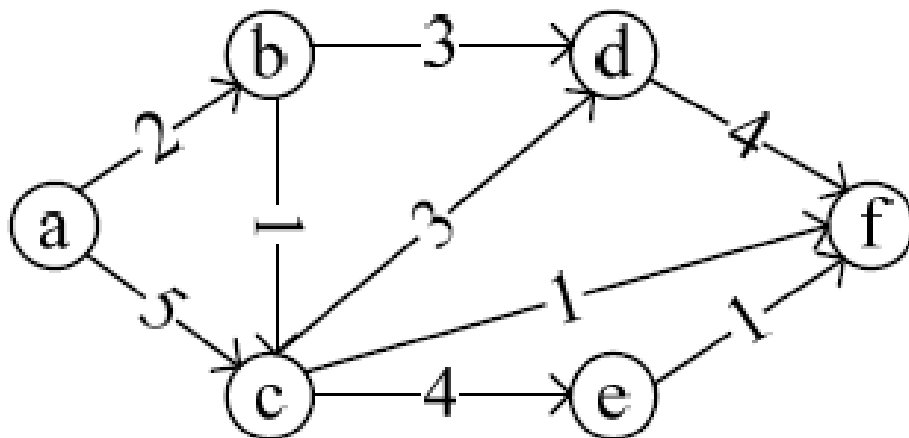
$$(v_0, v_2) + (v_2, v_3) < (v_0, v_3)$$

与最小生成树的不同点：路径可能是累加的！

最短路径——迪杰斯特拉算法

如下有向带权图,若采用迪杰斯特拉算法求源点a到其他各顶点的最短路径,得到的第一条最短路径的目标顶点是b,第二条最短路径的目标顶点是c,后续得到的其余各最短路径的目标顶点依次是()

- A **d,e,f**
- B **e,d,f**
- C **f,d,e**
- D **f,e,d**



最短路径——迪杰斯特拉算法

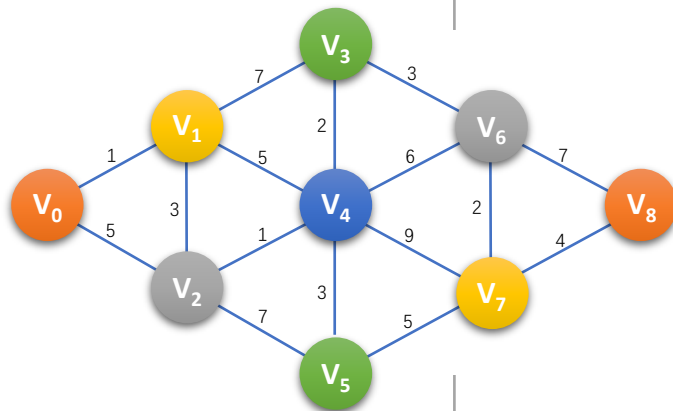
关键代码

```
void ShortestPath_Dijkstra(MGraph G, int v0, Patharc &P, ShortPathTable &D){
```

```
    int v,w,k,min;
    int final[MAXVEX];
    for(v=0; v<G.numVertexes; v++) {
        final[v] = 0;
        D[v] = G.arc[v0][v];
        P[v] = -1;
    }

    D[v0] = 0;
    final[v0] = 1;
    for(v=1; v<G.numVertexes; v++) {
        min=GRAPH_INFINITY;
        for(w=0; w<G.numVertexes; w++) {
            if(!final[w] && D[w]<min) {
                k=w;
                min = D[w];
            }
        }
        final[k] = 1;
        for(w=0; w<G.numVertexes; w++) {
            if(!final[w] && (min+G.arc[k][w]<D[w])) {
                D[w] = min + G.arc[k][w];
                P[w]=k;
            }
        }
    }
}
```

数据变化



	V ₀	V ₁	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈
V ₀	0	1	5	∞	∞	∞	∞	∞	∞
V ₁	1	0	3	7	5	∞	∞	∞	∞
V ₂	5	3	0	∞	1	7	∞	∞	∞
V ₃	∞	7	∞	0	2	∞	3	∞	∞
V ₄	∞	5	1	2	0	3	6	9	∞
V ₅	∞	∞	7	∞	3	0	∞	5	∞
V ₆	∞	∞	∞	3	6	∞	0	2	7
V ₇	∞	∞	∞	∞	9	5	2	0	4
V ₈	∞	∞	∞	∞	∞	∞	7	4	0

final	D	P
1	0	-1
0	1	-1
0	5	-1
0	65535	-1
0	65535	-1
0	65535	-1
0	65535	-1
0	65535	-1
0	65535	-1
0	65535	-1

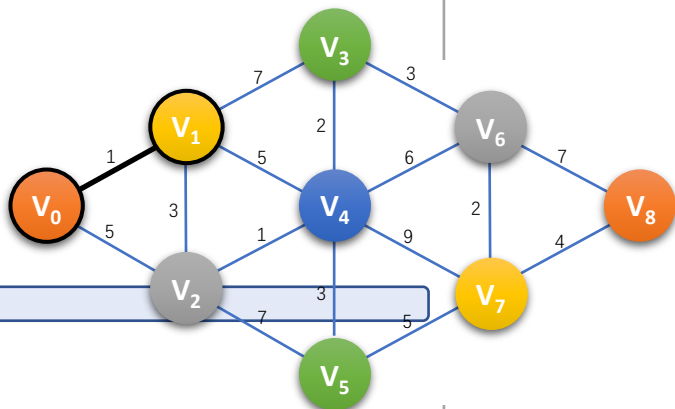
最短路径——迪杰斯特拉算法

关键代码

```
void ShortestPath_Dijkstra(MGraph G, int v0, Patharc &P, ShortPathTable &D){
    int v,w,k,min;
    int final[MAXVEX];
    for(v=0; v<G.numVertexes; v++) {
        final[v] = 0;
        D[v] = G.arc[v0][v];
        P[v] = -1;
    }

    D[v0] = 0;
    final[v0] = 1;
    for(v=1; v<G.numVertexes; v++) {
        min=GRAPH_INFINITY;
        for(w=0; w<G.numVertexes; w++) {
            if(!final[w] && D[w]<min) {
                k=w;
                min = D[w];
            }
        }
        final[k] = 1;
        for(w=0; w<G.numVertexes; w++) {
            if(!final[w] && (min+G.arc[k][w]<D[w])) {
                D[w] = min + G.arc[k][w];
                P[w]=k;
            }
        }
    }
}
```

数据变化



	V ₀	V ₁	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈
V ₀	0	1	5	∞	∞	∞	∞	∞	∞
V ₁	1	0	3	7	5	∞	∞	∞	∞
V ₂	5	3	0	∞	1	7	∞	∞	∞
V ₃	∞	7	∞	0	2	∞	3	∞	∞
V ₄	∞	5	1	2	0	3	6	9	∞
V ₅	∞	∞	7	∞	3	0	∞	5	∞
V ₆	∞	∞	∞	3	6	∞	0	2	7
V ₇	∞	∞	∞	∞	9	5	2	0	4
V ₈	∞	∞	∞	∞	∞	∞	7	4	0

final	D	P
1	0	-1
1	1	-1
0	5	-1
0	65535	-1
0	65535	-1
0	65535	-1
0	65535	-1
0	65535	-1
0	65535	-1
0	65535	-1

v=1
 w=0~9
 min=1
 k=1
 final[k]=final[1]=1

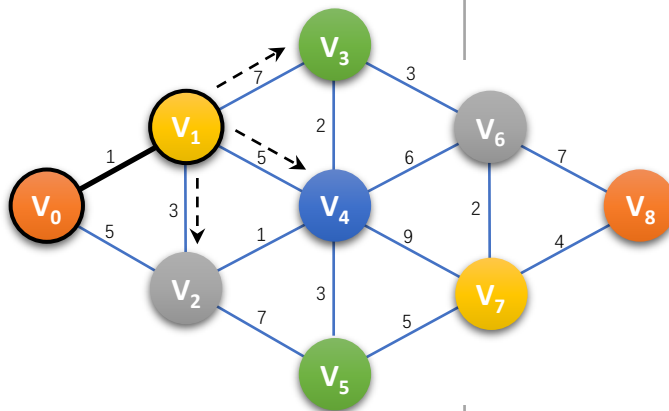
最短路径——迪杰斯特拉算法

关键代码

```
void ShortestPath_Dijkstra(MGraph G, int v0, Patharc &P, ShortPathTable &D){
    int v,w,k,min;
    int final[MAXVEX];
    for(v=0; v<G.numVertexes; v++){
        final[v] = 0;
        D[v] = G.arc[v0][v];
        P[v] = -1;
    }

    D[v0] = 0;
    final[v0] = 1;
    for(v=1; v<G.numVertexes; v++){
        min=GRAPH_INFINITY;
        for(w=0; w<G.numVertexes; w++){
            if(!final[w] && D[w]<min) {
                k=w;
                min = D[w];
            }
        }
        final[k] = 1;
        for(w=0; w<G.numVertexes; w++) {
            if(!final[w] && (min+G.arc[k][w]<D[w])) {
                D[w] = min + G.arc[k][w];
                P[w]=k;
            }
        }
    }
}
```

数据变化



	V ₀	V ₁	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈
V ₀	0	1	5	∞	∞	∞	∞	∞	∞
V ₁	1	0	3	7	5	∞	∞	∞	∞
V ₂	5	3	0	∞	1	7	∞	∞	∞
V ₃	∞	7	∞	0	2	∞	3	∞	∞
V ₄	∞	5	1	2	0	3	6	9	∞
V ₅	∞	∞	7	∞	3	0	∞	5	∞
V ₆	∞	∞	∞	3	6	∞	0	2	7
V ₇	∞	∞	∞	∞	9	5	2	0	4
V ₈	∞	∞	∞	∞	∞	∞	7	4	0

final	D	P
1	0	-1
1	1	-1
0	4	1
0	8	1
0	6	1
0	65535	-1
0	65535	-1
0	65535	-1
0	65535	-1

$v=1$
 $w=0\sim 9$
 $\min=1$
 $k=1$
 $\text{final}[k]=\text{final}[1]=1$

$v_0-v_1-v_2: 1+3=4$
 $v_0-v_1-v_3: 1+7=8$
 $v_0-v_1-v_4: 1+5=6$

最短路径——迪杰斯特拉算法

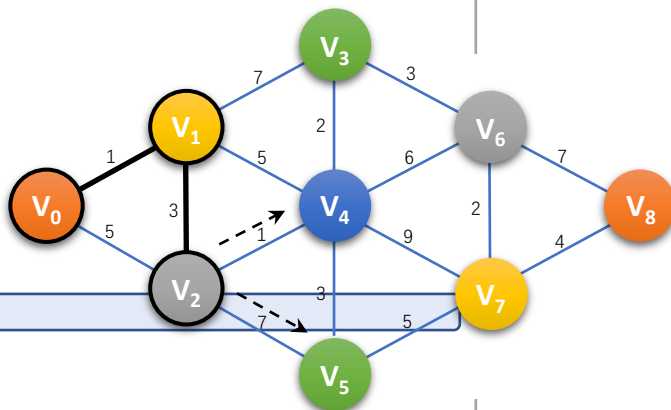
关键代码

```
void ShortestPath_Dijkstra(MGraph G, int v0, Patharc &P, ShortPathTable &D){
    int v,w,k,min;
    int final[MAXVEX];
    for(v=0; v<G.numVertexes; v++) {
        final[v] = 0;
        D[v] = G.arc[v0][v];
        P[v] = -1;
    }
}
```

```
D[v0] = 0;
final[v0] = 1;
```

```
for(v=1; v<G.numVertexes; v++) {
    min=GRAPH_INFINITY;
    for(w=0; w<G.numVertexes; w++) {
        if(!final[w] && D[w]<min) {
            k=w;
            min = D[w];
        }
    }
    final[k] = 1;
    for(w=0; w<G.numVertexes; w++) {
        if(!final[w] && (min+G.arc[k][w]<D[w])) {
            D[w] = min + G.arc[k][w];
            P[w]=k;
        }
    }
}
```

数据变化



	V ₀	V ₁	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈
V ₀	0	1	5	∞	∞	∞	∞	∞	∞
V ₁	1	0	3	7	5	∞	∞	∞	∞
V ₂	5	3	0	∞	1	7	∞	∞	∞
V ₃	∞	7	∞	0	2	∞	3	∞	∞
V ₄	∞	5	1	2	0	3	6	9	∞
V ₅	∞	∞	7	∞	3	0	∞	5	∞
V ₆	∞	∞	∞	3	6	∞	0	2	7
V ₇	∞	∞	∞	∞	9	5	2	0	4
V ₈	∞	∞	∞	∞	∞	∞	7	4	0

final	D	P
1	0	-1
1	1	-1
1	4	1
0	8	1
0	5	2
0	11	2
0	65535	-1
0	65535	-1
0	65535	-1

v=2

w=0~9

min=4

k=2

final[k]=final[2]=1

V₀-V₁-V₂-V₄: 1+3+1=5

V₀-V₁-V₂-V₅: 1+3+7=11

最短路径——迪杰斯特拉算法

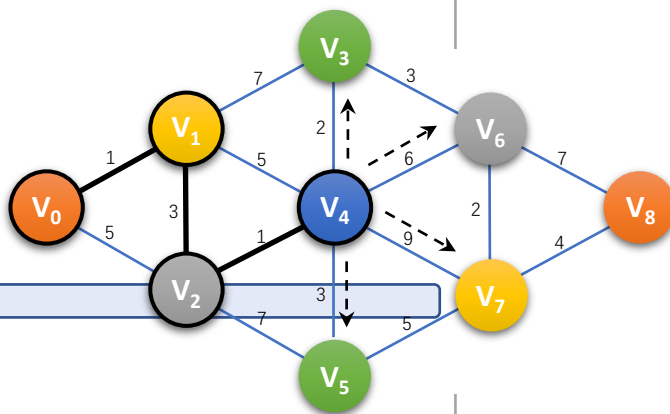
关键代码

```
void ShortestPath_Dijkstra(MGraph G, int v0, Patharc &P, ShortPathTable &D){
    int v,w,k,min;
    int final[MAXVEX];
    for(v=0; v<G.numVertexes; v++){
        final[v] = 0;
        D[v] = G.arc[v0][v];
        P[v] = -1;
    }
}
```

```
D[v0] = 0;
final[v0] = 1;
```

```
for(v=1; v<G.numVertexes; v++){
    min=GRAPH_INFINITY;
    for(w=0; w<G.numVertexes; w++){
        if(!final[w] && D[w]<min){
            k=w;
            min = D[w];
        }
    }
    final[k] = 1;
    for(w=0; w<G.numVertexes; w++){
        if(!final[w] && (min+G.arc[k][w]<D[w])) {
            D[w] = min + G.arc[k][w];
            P[w]=k;
        }
    }
}
```

数据变化



	V ₀	V ₁	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈
V ₀	0	1	5	∞	∞	∞	∞	∞	∞
V ₁	1	0	3	7	5	∞	∞	∞	∞
V ₂	5	3	0	∞	1	7	∞	∞	∞
V ₃	∞	7	∞	0	2	∞	3	∞	∞
V ₄	∞	5	1	2	0	3	6	9	∞
V ₅	∞	∞	7	∞	3	0	∞	5	∞
V ₆	∞	∞	∞	3	6	∞	0	2	7
V ₇	∞	∞	∞	∞	9	5	2	0	4
V ₈	∞	∞	∞	∞	∞	∞	7	4	0

	final	D	P
1	1	0	-1
1	1	1	-1
1	1	4	1
0	0	7	4
1	1	5	2
0	0	8	4
0	0	11	4
0	0	14	4
0	0	65535	-1

v=4

w=0~9

min=5

k=4

final[k]=final[4]=1

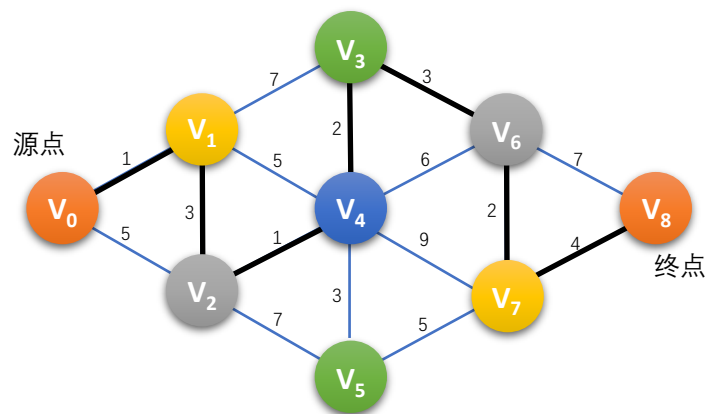
$V_0-V_1-V_2-V_4-V_3$: $1+3+1+2=7$

$V_0-V_1-V_2-V_4-V_5$: $1+3+1+3=8$

$V_0-V_1-V_2-V_4-V_6$: $1+3+1+6=11$

$V_0-V_1-V_2-V_4-V_7$: $1+3+1+9=14$

最短路径——迪杰斯特拉算法



最短路径——迪杰斯特拉算法

(1) **A[n][n]**: 有向网带权邻接矩阵, **A[i][j]**表示弧 (v_i, v_j)的权值; **S**: 已找到从 v_0 出发的最短路径的终点集合, 它的初始状态为{ v_0 }; **D[n]**: v_0 到各终点当前找到的最短路径长度, 它的初始值**D[i] = A[v₀,i]** ;

(2) 选择 j, 使得 **D[j] = min{D[i] | $v_i \in V-S$ }**
/* v_i : S之外的顶点, j: 当前离 v_0 距离最短的顶点, D[j]: 从 v_0 到**S集合之外所有顶点**的路径中距离最短的一条*/

S = S $\cup$ { v_j } //将j加入S集

最短路径——迪杰斯特拉算法

(3) 对于所有不在S中的终点 v_k , 若

$$D[j] + A[j,k] < D[k] \quad // \text{即}(v_0,j) + (j,k) < (v_0,k)$$

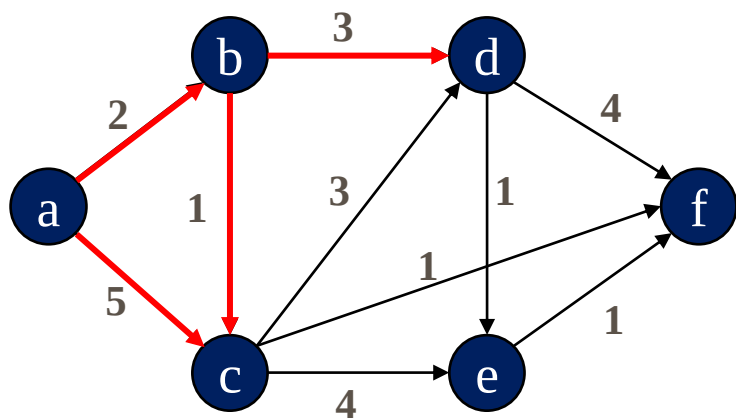
则修改D[k]为: $D[k] = D[j] + A[j,k]$

//D[j]为上次求得从源点到 v_j 点的最短路径, v_j 为上次求得最短路径的点。

(4) 重复操作(2)、(3)共n-1次, 由此求得从 v_0 到各终点的最短路径。

最短路径——迪杰斯特拉算法

求a到其他点的最短路径



	a	b	c	d	e	f
a	0	2	5	∞	∞	∞
b	∞	0	1	3	∞	∞
c	∞	∞	0	3	4	1
d	∞	∞	∞	0	1	4
e	∞	∞	∞	∞	0	1
f	∞	∞	∞	∞	∞	0

每次选距离a最近的点,然后更新

	第1次	第2次	第3次	第4次	第5次
b	2 (a,b)				
c	5 (a,c)	3 (a,b,c)			
d	∞	5 (a,b,d)	5 (a,b,d)	5 (a,b,d)	
e	∞	∞	7 (a,b,c,e)	7 (a,b,c,e)	6 (a,b,d,e)
f	∞	∞	4 (a,b,c,f)		
终点集	{a,b}	{a,b,c}	{a,b,c,f}	{a,b,c,f,d}	{a,b,c,f,d,e}

$D[w]$

$(a,b) + (b,c) < (a,c)$

S

最短路径——迪杰斯特拉算法

```
void ShortestPath_Dijkstra(MGraph G, int v0, Patharc &P, ShortPathTable &D){
    int v,w,k,min;
    int final[MAXVEX];
    for(v=0; v<G.vexnum; v++) {
        final[v] = 0; D[v] = G.arc[v0][v]; P[v] = -1;
    }
    D[v0] = 0;
    final[v0] = 1; // 初始化, v0顶点属于S集
    for(v=1; v<G.vexnum; v++) { // 其余G.vexnum-1 个顶点
        min=INFINITY;
        for(w=0; w<G.vexnum; w++) {
            if(!final[w] && D[w]<min) { // w顶点在V-S中, 且w顶点离v0更近
                k = w; min = D[w];
            }
        }
        final[k] = 1; // 离v0 顶点最近的v加入S集
        for(w=0; w<G.vexnum; w++) { // 更新当前最短路径及距离
            if(!final[w] && (min+G.arc[k][w]<D[w])) { // 修改D[w]和P[w], w∈V- S
                D[w] = min + G.arc[k][w];
                P[w] = k;
            }
        }
    }
}
```

最短路径——弗洛伊德算法

问题的提出：已知一个各边权值均大于0的带权有向图，对每一对顶点 $v_i \neq v_j$ ，希望求出 v_i 与 v_j 之间的最短路径和最短路径长度。

解决思路：

可以通过调用n次Dijkstra算法来完成，但时间复杂度为 $O(n^3)$ 。

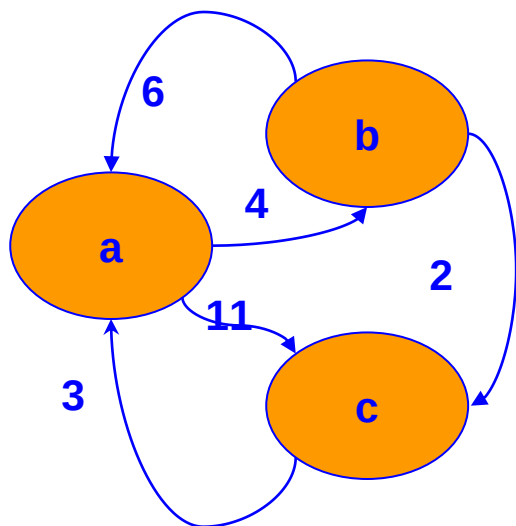
改进：Floyd算法

最短路径——弗洛伊德算法

从演示中看算法思想

一个简单的图及其邻接矩阵如下：

D(-1)



	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cb, ∞)	(cc,0)

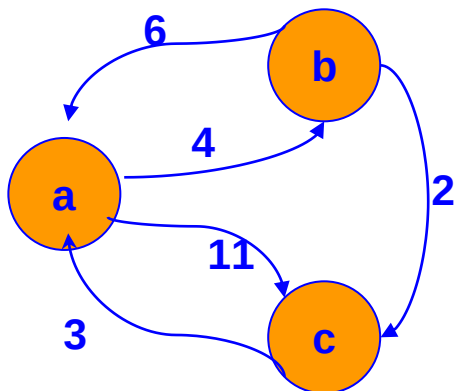
最短路径——弗洛伊德算法

从上面的 $D^{(-1)}$ 开始，对于每两个顶点 u 、 v ，在 $D^{(-1)}$ 中存储着一条路径 $u\dots v$ 。现在我们考察，试着把 a 加到 u 、 v 的路径上能否，得到一条更短的路径，即如果 $u\dots a + a\dots v < u\dots v$ 的话，能够找到一条更短的路径。

$$D^{(k)}[i][j] = \text{Min}\{D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j]\}$$

依次将每个点作为“中间点”去做更新

最短路径——弗洛伊德算法



本来路径上源点或终点就有a的不必考虑。

对角线上的也不必考虑

$D^{(-1)}$

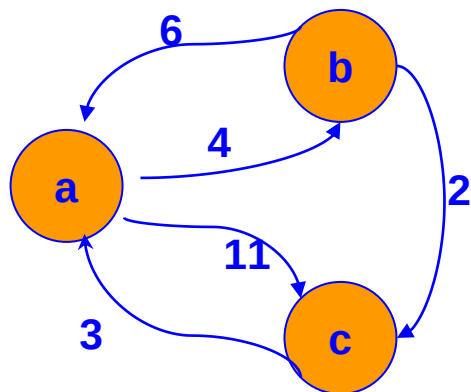
	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cb, ∞)	(cc,0)



$D^{(0)}$

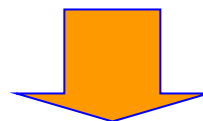
	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	
c	(ca,3)		(cc,0)

最短路径——弗洛伊德算法



D⁽⁻¹⁾

	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cb, ∞)	(cc,0)

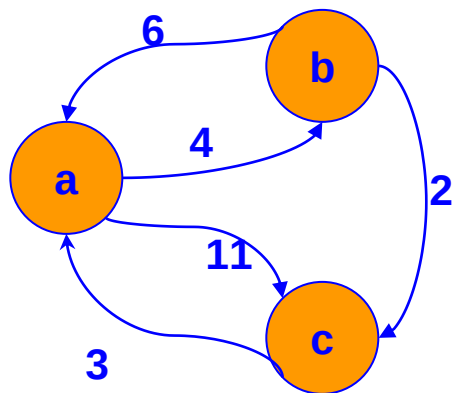


$D[b][a] + D[a][c]$
 $= 6 + 11 > D[b][c]$
 $= 2$, 所以如果
 从a绕, 反而远,
 那么这一项不变。

D⁽⁰⁾

	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	
c	(ca,3)		(cc,0)

最短路径——弗洛伊德算法



D⁽⁻¹⁾

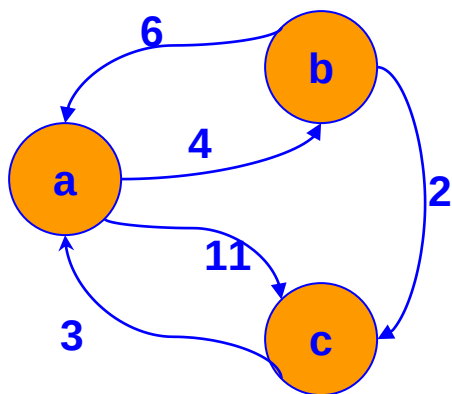
	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cb, ∞)	(cc,0)



D⁽⁰⁾

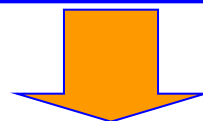
	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)		(cc,0)

最短路径——弗洛伊德算法



$D^{(-1)}$

	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cb, ∞)	(cc,0)

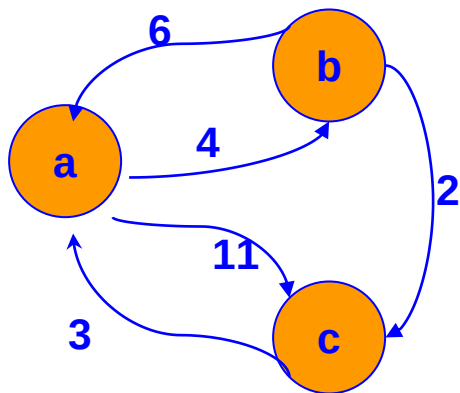


$D[c][a] + D[a][b]$
 $= 3 + 4 < D[c][b]$
 $= \infty$, 所以如果
从a绕, 更近,
那么应该更新。

$D^{(0)}$

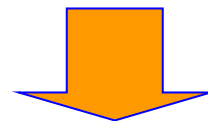
	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)		(cc,0)

最短路径——弗洛伊德算法



$D^{(-1)}$

	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cb, ∞)	(cc,0)



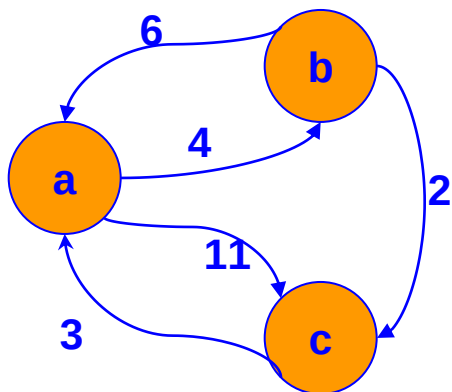
$D^{(0)}$

	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)

最短路径——弗洛伊德算法

从上面的 $D^{(0)}$ 开始，对于每两个顶点 u 、 v ，在 $D^{(0)}$ 中存储着一条路径 $u...v$ 。现在我们考察，试着把 b 加到 u 、 v 的路径上能否，得到一条更短的路径，即如果 $u...b + b...v < u...v$ 的话，能够找到一条更短的路径。

最短路径——弗洛伊德算法



本来路径上源点
或终点就有b的不
必考虑。

对角线上的也不
必考虑

$D^{(0)}$

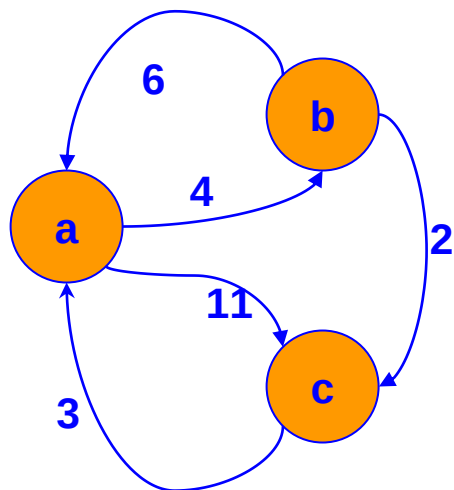
	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)



$D^{(1)}$

	a	b	c
a	(aa,0)	(ab,4)	
b	(ba,6)	(bb,0)	(bc,2)
c		(cab,7)	(cc,0)

最短路径——弗洛伊德算法



$D[a][b] + D[b][c] = 6 < D[a][c]$, 所以
如果从b绕, 更近,
那么应该更新。

$D^{(0)}$

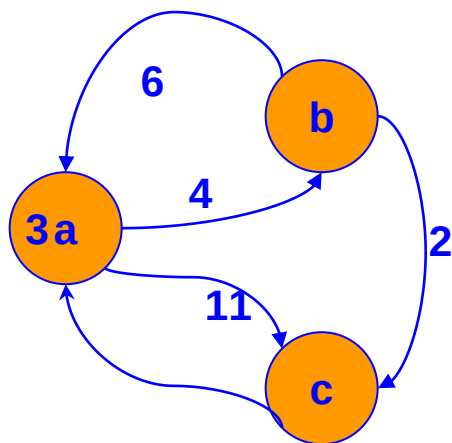
	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)



$D^{(1)}$

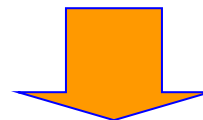
	a	b	c
a	(aa,0)	(ab,4)	
b	(ba,6)	(bb,0)	(bc,2)
c		(cab,7)	(cc,0)

最短路径——弗洛伊德算法



$D^{(0)}$

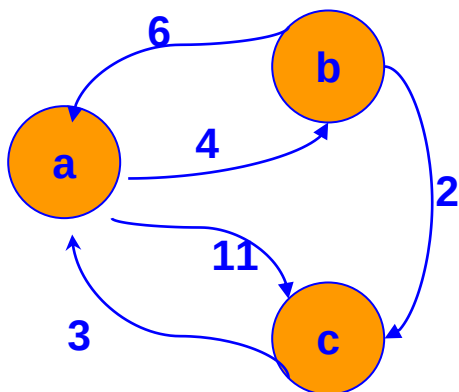
	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)



$D^{(1)}$

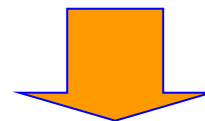
	a	b	c
a	(aa,0)	(ab,4)	(abc,6)
b	(ba,6)	(bb,0)	(bc,2)
c		(cab,7)	(cc,0)

最短路径——弗洛伊德算法



$D^{(0)}$

	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)

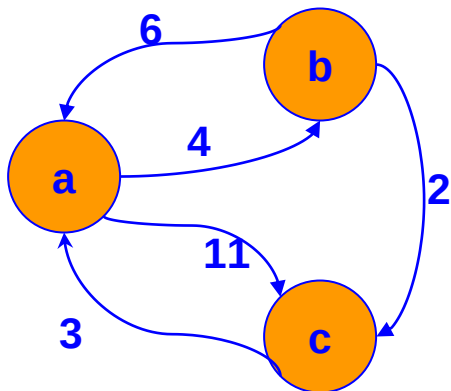


$D^{(1)}$

	a	b	c
a	(aa,0)	(ab,4)	(abc,6)
b	(ba,6)	(bb,0)	(bc,2)
c		(cab,7)	(cc,0)

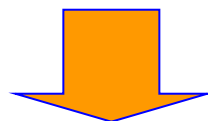
$D[c][b] + D[b][a] = 7 + 6 > D[c][a] = 3$, 所以如果从b绕, 反而远, 那么这一项不变。

最短路径——弗洛伊德算法



$D^{(0)}$

	a	b	c
a	(aa,0)	(ab,4)	(ac,11)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)



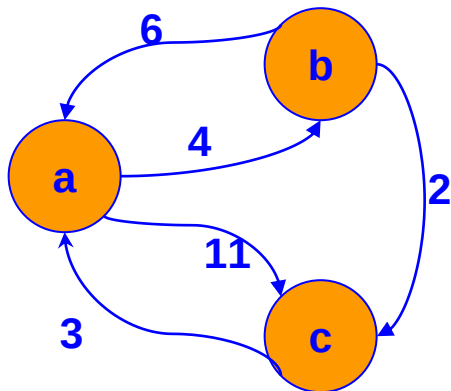
$D^{(1)}$

	a	b	c
a	(aa,0)	(ab,4)	(abc,6)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)

最短路径——弗洛伊德算法

从上面的 $D^{(1)}$ 开始，对于每两个顶点 u 、 v ，在 $D^{(1)}$ 中存储着一条路径 $u\dots v$ 。现在我们考察，试着把 c 加到 u 、 v 的路径上能否，得到一条更短的路径，即如果 $u\dots c + c\dots v < u\dots v$ 的话，能够找到一条更短的路径。

最短路径——弗洛伊德算法



本来路径上源点或
终点就有c的不必考
虑。

对角线上的也不必
考虑

$D^{(1)}$

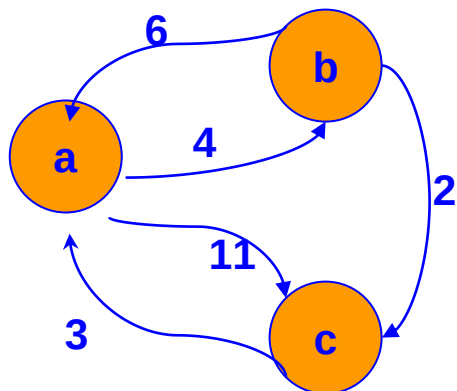
	a	b	c
a	(aa,0)	(ab,4)	(abc,6)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)



$D^{(2)}$

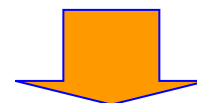
	a	b	c
a	(aa,0)		(abc,6)
b		(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)

最短路径——弗洛伊德算法



$D^{(1)}$

	a	b	c
a	(aa,0)	(ab,4)	(abc,6)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)

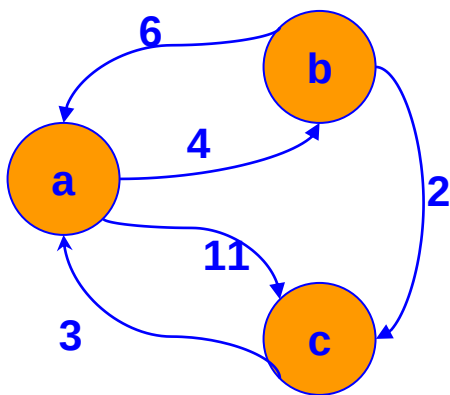


$D[a][c] + D[c][b] = 6 + 7 > D[a][b] = 4$,
所以如果从c绕,
反而远, 那么这一
项不变。

$D^{(2)}$

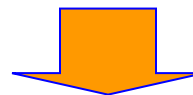
	a	b	c
a	(aa,0)		(abc,6)
b		(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)

最短路径——弗洛伊德算法



$D^{(1)}$

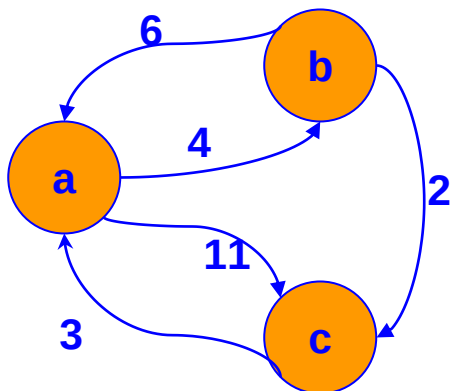
	a	b	c
a	(aa,0)	(ab,4)	(abc,6)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)



$D^{(2)}$

	a	b	c
a	(aa,0)	(ab,4)	(abc,6)
b		(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)

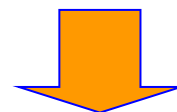
最短路径——弗洛伊德算法



$D[b][c] + D[c][a] =$
 $2 + 3 < D[b][a] = 6,$
所以如果从c绕,
更近, 那么应该更
新。

$D^{(1)}$

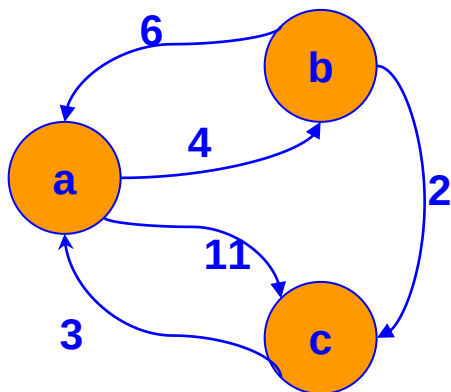
	a	b	c
a	(aa,0)	(ab,4)	(abc,6)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)



$D^{(2)}$

	a	b	c
a	(aa,0)	(ab,4)	(abc,6)
b		(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)

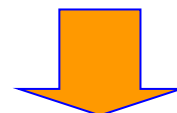
最短路径——弗洛伊德算法



$D[b][c] + D[c][a]$
 $= 2 + 3 < D[b][a] =$
5, 所以如果从c
绕, 更近, 那么
应该更新。

$D^{(1)}$

	a	b	c
a	(aa,0)	(ab,4)	(abc,6)
b	(ba,6)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)



$D^{(2)}$

	a	b	c
a	(aa,0)	(ab,4)	(abc,6)
b	(bca,5)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)

最短路径——弗洛伊德算法

现在，已经把所有的顶点都试了一遍，算法结束。
每两个顶点之间的路径如 $D^{(2)}$ 所示。

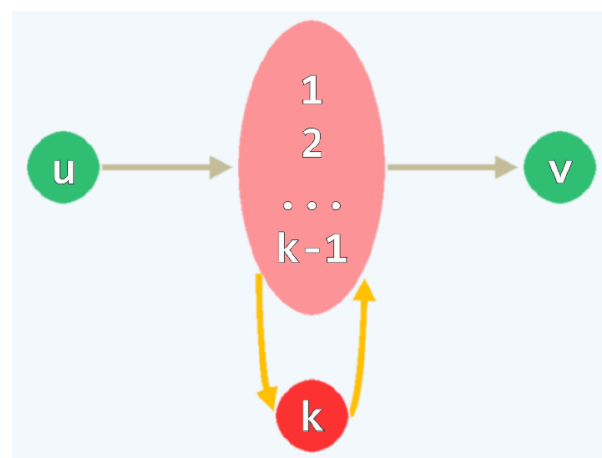
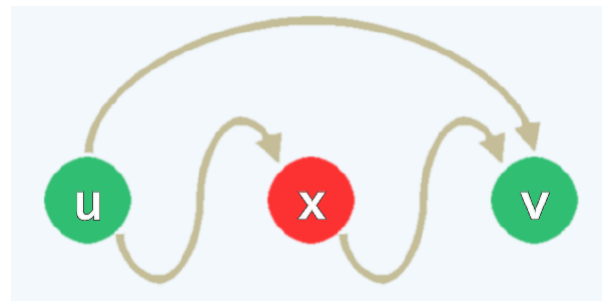
$D^{(2)}$

	a	b	c
a	(aa,0)	(ab,4)	(abc,6)
b	(bca,5)	(bb,0)	(bc,2)
c	(ca,3)	(cab,7)	(cc,0)

最短路径——弗洛伊德算法

□ u 和 v 之间的最短路径可能是

- 不存在通路, 或者
- 直接连接, 或者
- 最短路径(u, x) + 最短路径(x, v)



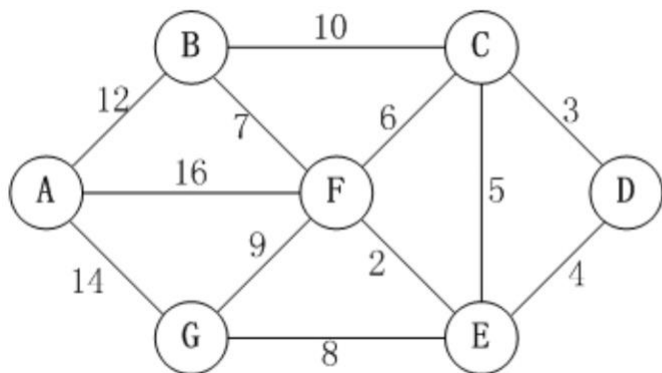
□ 将所有顶点随意编号: 1, 2, .., n

□ 定义:

$$\begin{aligned} d^k(u, v) &= \text{中途只经过前} k \text{个顶点、联接} u \text{和} v \text{的最短路径长度} \\ &= w(u, v) && (\text{if } k = 0) \\ &= \min(d^{k-1}(u, v), d^{k-1}(u, k) + d^{k-1}(k, v)) && (\text{if } k \geq 1) \end{aligned}$$

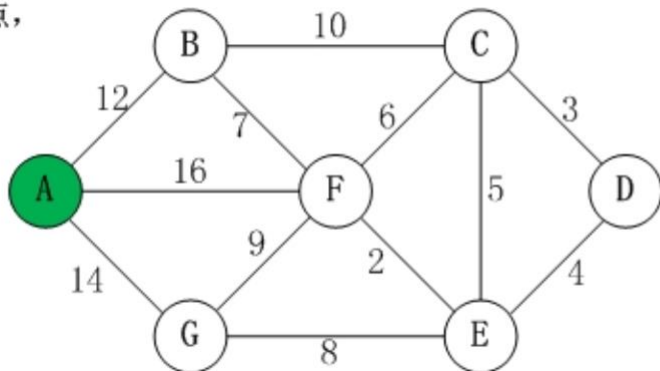
最短路径——弗洛伊德算法

第1步：
初始化矩阵S



	A	B	C	D	E	F	G
A	0	12	INF	INF	INF	16	14
B	12	0	10	INF	INF	7	INF
C	INF	10	0	3	5	6	INF
D	INF	INF	3	0	4	INF	INF
E	INF	INF	5	4	0	2	8
F	16	7	6	INF	2	0	9
G	14	INF	INF	INF	8	9	0

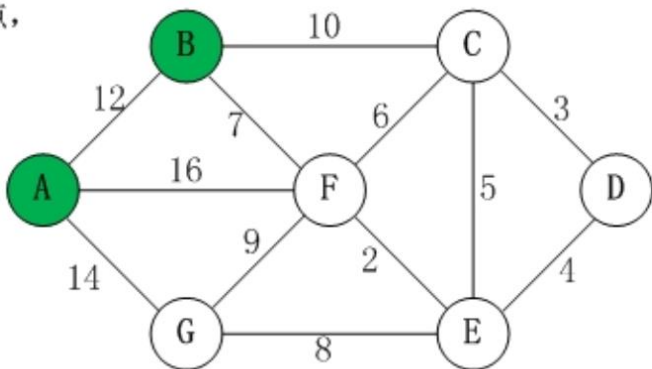
第2步：
以顶点A为中介点，
更新矩阵S。



	A	B	C	D	E	F	G
A	0	12	INF	INF	INF	16	14
B	12	0	10	INF	INF	7	26
C	INF	10	0	3	5	6	INF
D	INF	INF	3	0	4	INF	INF
E	INF	INF	5	4	0	2	8
F	16	7	6	INF	2	0	9
G	14	26	INF	INF	8	9	0

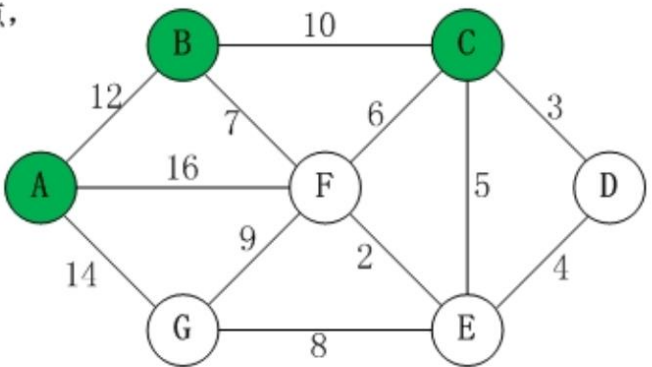
最短路径——弗洛伊德算法

第3步：
以顶点B为中介点，
更新矩阵S。



	A	B	C	D	E	F	G
A	0	12	22	INF	INF	16	14
B	12	0	10	INF	INF	7	26
C	22	10	0	3	5	6	36
D	INF	INF	3	0	4	INF	INF
E	INF	INF	5	4	0	2	8
F	16	7	6	INF	2	0	9
G	14	26	36	INF	8	9	0

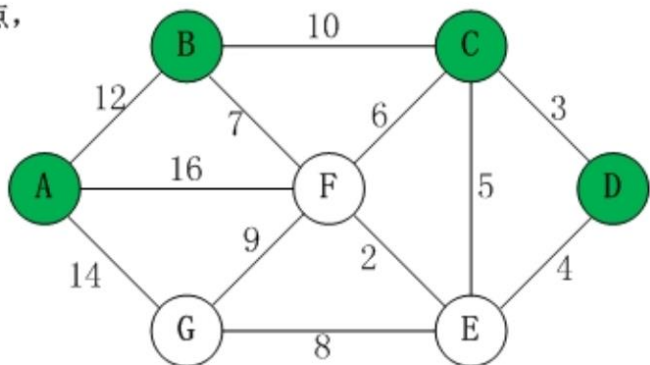
第4步：
以顶点C为中介点，
更新矩阵S。



	A	B	C	D	E	F	G
A	0	12	22	25	27	16	14
B	12	0	10	13	15	7	26
C	22	10	0	3	5	6	36
D	25	13	3	0	4	9	39
E	27	15	5	4	0	2	8
F	16	7	6	9	2	0	9
G	14	26	36	39	8	9	0

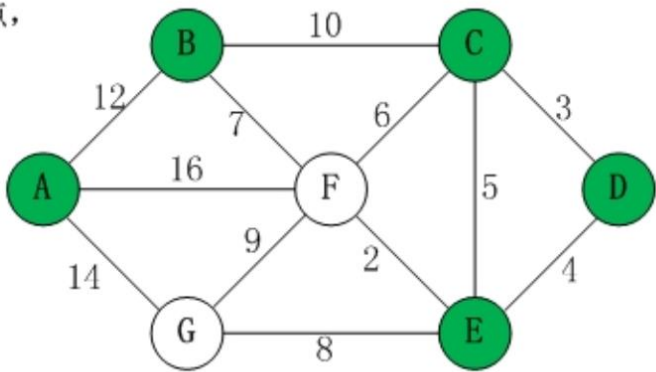
最短路径——弗洛伊德算法

第5步：
以顶点D为中介点，
更新矩阵S。



	A	B	C	D	E	F	G
A	0	12	22	25	27	16	14
B	12	0	10	13	15	7	26
C	22	10	0	3	5	6	36
D	25	13	3	0	4	9	39
E	27	15	5	4	0	2	8
F	16	7	6	9	2	0	9
G	14	26	36	39	8	9	0

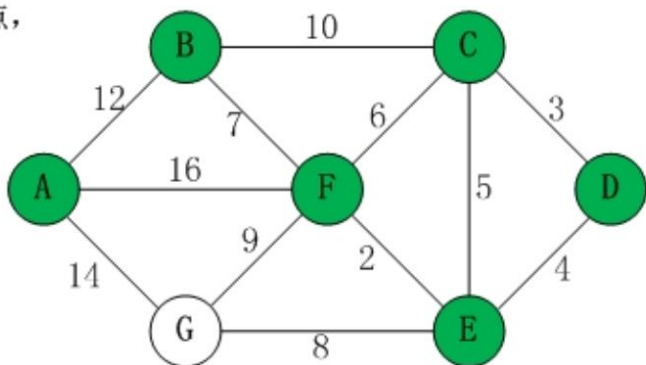
第6步：
以顶点E为中介点，
更新矩阵S。



	A	B	C	D	E	F	G
A	0	12	22	25	27	16	14
B	12	0	10	13	15	7	23
C	22	10	0	3	5	6	13
D	25	13	3	0	4	6	12
E	27	15	5	4	0	2	8
F	16	7	6	6	2	0	9
G	14	23	13	12	8	9	0

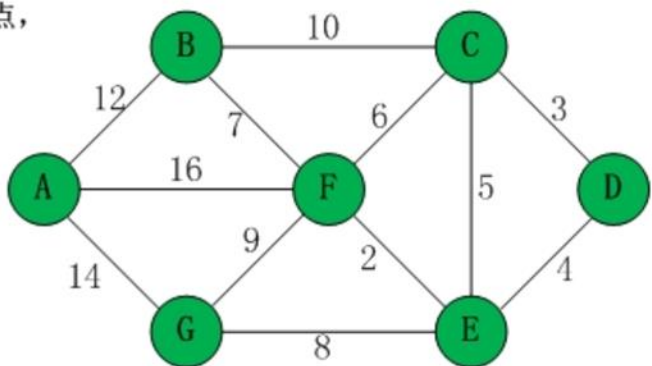
最短路径——弗洛伊德算法

第7步：
以顶点F为中介点，
更新矩阵S。



	A	B	C	D	E	F	G
A	0	12	22	22	18	16	14
B	12	0	10	13	9	7	16
C	22	10	0	3	5	6	13
D	22	13	3	0	4	6	12
E	18	9	5	4	0	2	8
F	16	7	6	6	2	0	9
G	14	16	13	12	8	9	0

第8步：
以顶点G为中介点，
更新矩阵S。



	A	B	C	D	E	F	G
A	0	12	22	22	18	16	14
B	12	0	10	13	9	7	16
C	22	10	0	3	5	6	13
D	22	13	3	0	4	6	12
E	18	9	5	4	0	2	8
F	16	7	6	6	2	0	9
G	14	16	13	12	8	9	0

最短路径——弗洛伊德算法

```
void ShortestPath_Floyd(MGraph G, Patharc &P, ShortPathTable &D)
```

```
{
    int v,w,k;
    k=0
```

```
    for(v=0; v<G.vexnum; ++v)
```

```
    {
        for(w=0; w<G.vexnum; ++w)
```

```
        {
            D[v][w]=G.arc[v][w];
            P[v][w]=w;
```

```
        }
    }
    for(k=0; k<G.vexnum; ++k)
```

```
    {
        for(v=0; v<G.vexnum; ++v)
```

```
        {
            for(w=0; w<G.vexnum; ++w)
```

```
            {
                if (D[v][w]>D[v][k]+D[k][w])
```

```
                {
                    D[v][w]=D[v][k]+D[k][w];
                    P[v][w]=P[v][k];
```

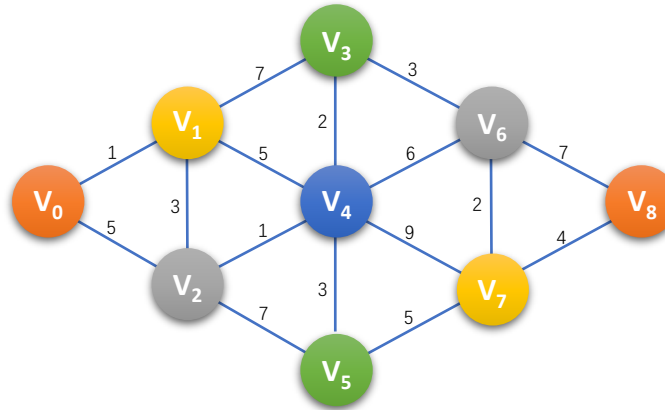
```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
}
```



	V ₀	V ₁	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈
V ₀	0	1	5	∞	∞	∞	∞	∞	∞
V ₁	1	0	3	7	5	∞	∞	∞	∞
V ₂	5	3	0	∞	1	7	∞	∞	∞
V ₃	∞	7	∞	0	2	∞	3	∞	∞
V ₄	∞	5	1	2	0	3	6	9	∞
V ₅	∞	∞	7	∞	3	0	∞	5	∞
V ₆	∞	∞	∞	3	6	∞	0	2	7
V ₇	∞	∞	∞	∞	9	5	2	0	4
V ₈	∞	∞	∞	∞	∞	∞	7	4	0

D⁰

	V ₀	V ₁	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈
V ₀	0	1	5	∞	∞	∞	∞	∞	∞
V ₁	1	0	3	7	5	∞	∞	∞	∞
V ₂	5	3	0	∞	1	7	∞	∞	∞
V ₃	∞	7	∞	0	2	∞	3	∞	∞
V ₄	∞	5	1	2	0	3	6	9	∞
V ₅	∞	∞	7	∞	3	0	∞	5	∞
V ₆	∞	∞	∞	3	6	∞	0	2	7
V ₇	∞	∞	∞	∞	9	5	2	0	4
V ₈	∞	∞	∞	∞	∞	∞	7	4	0

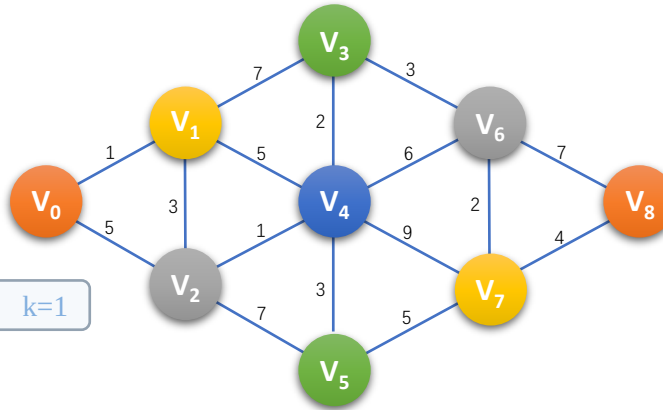
P⁰

	V ₀	V ₁	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈
V ₀	0	1	2	3	4	5	6	7	8
V ₁	0	1	2	3	4	5	6	7	8
V ₂	0	1	2	3	4	5	6	7	8
V ₃	0	1	2	3	4	5	6	7	8
V ₄	0	1	2	3	4	5	6	7	8
V ₅	0	1	2	3	4	5	6	7	8
V ₆	0	1	2	3	4	5	6	7	8
V ₇	0	1	2	3	4	5	6	7	8
V ₈	0	1	2	3	4	5	6	7	8

最短路径——弗洛伊德算法

```
void ShortestPath_Floyd(MGraph G, Patharc &P, ShortPathTable &D)
```

```
{
    int v,w,k;
    for(v=0; v<G.vexnum; ++v)
    {
        for(w=0; w<G.vexnum; ++w)
        {
            D[v][w]=G.arc[v][w];
            P[v][w]=w;
        }
    }
    for(k=0; k<G.vexnum; ++k)
    {
        for(v=0; v<G.vexnum; ++v)
        {
            for(w=0; w<G.vexnum; ++w)
            {
                if (D[v][w]>D[v][k]+D[k][w])
                {
                    D[v][w]=D[v][k]+D[k][w];
                    P[v][w]=P[v][k];
                }
            }
        }
    }
}
```



	V ₀	V ₁	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈
V ₀	0	1	5	∞	∞	∞	∞	∞	∞
V ₁	1	0	3	7	5	∞	∞	∞	∞
V ₂	5	3	0	∞	1	7	∞	∞	∞
V ₃	∞	7	∞	0	2	∞	3	∞	∞
V ₄	∞	5	1	2	0	3	6	9	∞
V ₅	∞	∞	7	∞	3	0	∞	5	∞
V ₆	∞	∞	∞	3	6	∞	0	2	7
V ₇	∞	∞	∞	∞	9	5	2	0	4
V ₈	∞	∞	∞	∞	∞	∞	7	4	0

D¹

	v ₀	v ₁	v ₂	v ₃	v ₄	v ₅	v ₆	v ₇	v ₈
v ₀	0	1	4	8	6	∞	∞	∞	∞
v ₁	1	0	3	7	5	∞	∞	∞	∞
v ₂	4	3	0	10	1	7	∞	∞	∞
v ₃	8	7	10	0	2	∞	3	∞	∞
v ₄	6	5	1	2	0	3	6	9	∞
v ₅	∞	∞	7	∞	3	0	∞	5	∞
v ₆	∞	∞	∞	3	6	∞	0	2	7
v ₇	∞	∞	∞	∞	9	5	2	0	4
v ₈	∞	∞	∞	∞	∞	∞	7	4	0

P¹

	v ₀	v ₁	v ₂	v ₃	v ₄	v ₅	v ₆	v ₇	v ₈
v ₀	0	1	1	1	1	5	6	7	8
v ₁	0	1	2	3	4	5	6	7	8
v ₂	1	1	2	1	4	5	6	7	8
v ₃	1	1	1	3	4	5	6	7	8
v ₄	1	1	2	3	4	5	6	7	8
v ₅	0	1	2	3	4	5	6	7	8
v ₆	0	1	2	3	4	5	6	7	8
v ₇	0	1	2	3	4	5	6	7	8
v ₈	0	1	2	3	4	5	6	7	8

最短路径——弗洛伊德算法

```
void ShortestPath_Floyd(MGraph G, Patharc &P, ShortPathTable &D)
```

```
{
```

```
    int v,w,k;
```

```
    for(v=0; v<G.vexnum; ++v)
```

```
    {
```

```
        for(w=0; w<G.vexnum; ++w)
```

```
        {
```

```
            D[v][w]=G.arc[v][w];
```

```
            P[v][w]=w;
```

```
        }
```

```
    }
```

```
    for(k=0; k<G.vexnum; ++k)
```

k=2

```
    {
```

```
        for(v=0; v<G.vexnum; ++v)
```

```
        {
```

```
            for(w=0; w<G.vexnum; ++w)
```

```
            {
```

```
                if (D[v][w]>D[v][k]+D[k][w])
```

```
                {
```

```
                    D[v][w]=D[v][k]+D[k][w];
```

```
                    P[v][w]=P[v][k];
```

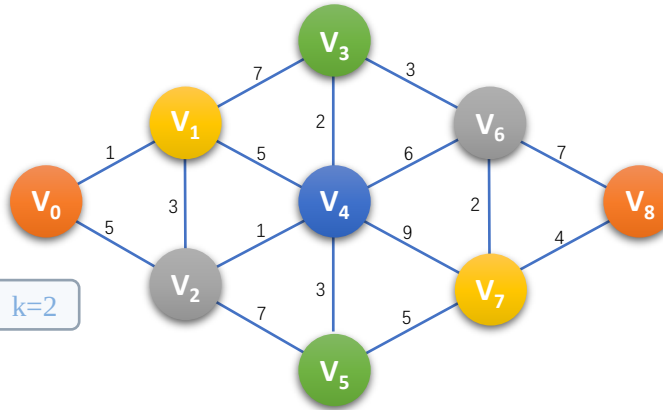
```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

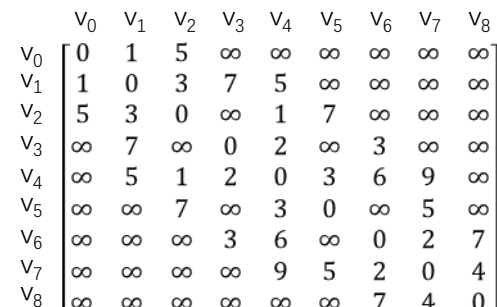


	V ₀	V ₁	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈
V ₀	0	1	5	∞	∞	∞	∞	∞	∞
V ₁	1	0	3	7	5	∞	∞	∞	∞
V ₂	5	3	0	∞	1	7	∞	∞	∞
V ₃	∞	7	∞	0	2	∞	3	∞	∞
V ₄	∞	5	1	2	0	3	6	9	∞
V ₅	∞	∞	7	∞	3	0	∞	5	∞
V ₆	∞	∞	∞	3	6	∞	0	2	7
V ₇	∞	∞	∞	∞	9	5	2	0	4
V ₈	∞	∞	∞	∞	∞	∞	7	4	0

D ²	V ₀	V ₁	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈
V ₀	0	1	4	8	5	11	∞	∞	∞
V ₁	1	0	3	7	4	10	∞	∞	∞
V ₂	4	3	0	10	1	7	∞	∞	∞
V ₃	8	7	10	0	2	17	3	∞	∞
V ₄	5	4	1	2	0	3	6	9	∞
V ₅	11	10	7	17	3	0	∞	5	∞
V ₆	∞	∞	∞	3	6	∞	0	2	7
V ₇	∞	∞	∞	∞	9	5	2	0	4
V ₈	∞	∞	∞	∞	∞	∞	7	4	0

P ²	V ₀	V ₁	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈
V ₀	0	1	1	1	1	1	6	7	8
V ₁	0	1	2	3	2	2	6	7	8
V ₂	1	1	2	1	4	5	6	7	8
V ₃	1	1	1	3	4	1	6	7	8
V ₄	2	2	2	3	4	5	6	7	8
V ₅	2	2	2	2	4	5	6	7	8
V ₆	0	1	2	3	4	5	6	7	8
V ₇	0	1	2	3	4	5	6	7	8
V ₈	0	1	2	3	4	5	6	7	8


```
{
    int v,w,k;
    for(v=0; v<G.vexnum; ++v)
    {
        for(w=0; w<G.vexnum; ++w)
        {
            D[v][w]=G.arc[v][w];
            P[v][w]=w;
        }
    }
    for(k=0; k<G.vexnum; ++k)
    {
        for(v=0; v<G.vexnum; ++v)
        {
            for(w=0; w<G.vexnum; ++w)
            {
                if (D[v][w]>D[v][k]+D[k][w])
                {
                    D[v][w]=D[v][k]+D[k][w];
                    P[v][w]=P[v][k];
                }
            }
        }
    }
}
```

[illegible]

最短路径——弗洛伊德算法

Floyd弗洛伊德算法时间复杂度也是 $O(n^3)$ ，但是形式上简单些。

正在答疑
